3D Computer Vision

CS 532 study textbook

Oct 22, 2026

CONTENTS

Before you begin 3

Notation and coordinate conventions 3
- Frames, axes, and transformations . . . 3
- Equality and homogeneous scale . . . 4
- Numerical checks . . . 5

Prerequisite refresher 6
- A quick readiness check . . . 6
- Matrix and vector tools used in Lecture 1 . . . 6
- Rank, nullspaces, and SVD . . . 7

I Lectures 1–6 8

1 Lecture 1: Foundations and 3D geometry 9
- 1.1 Why reconstruct a world from photographs? . . . 10
- 1.2 Points, lines, and the meaning of infinity . . . 14
- 1.3 Move points without losing the geometry . . . 18
- 1.4 Points, planes, and lines in 3D . . . 22
- 1.5 Rotations: several descriptions of the same motion . . . 28
- 1.6 Reference frames and two worked quiz problems . . . 33
- 1.7 Image formation: where does a 3D point appear? . . . 38
- 1.8 Foundation quiz practice . . . 45
- 1.9 Review: choose the right geometric operation . . . 47
- 1.10 TLDR . . . 50

2 Lecture 2: Projective geometry and image warping 53
- 2.1 Reading route . . . 53
- 2.2 Quaternions: convert without changing the rotation . . . 53
- 2.3 Homographies: straighten a photographed plane . . . 56
- 2.4 Image warping: where do the new pixels come from? . . . 59
- 2.5 Estimate a reliable mapping . . . 61
- 2.6 Distortion and two-view geometry . . . 65
- 2.7 Review and formula checklist . . . 69
- 2.8 TLDR . . . 70

3 Lecture 3: Camera projection, homographies, and RANSAC 72

3.1 Planned scope . . . 72
3.2 Examples and practice . . . 72
3.3 Completion checklist . . . 72

4 Lecture 4: Two-view geometry and stereo 73
4.1 Planned scope . . . 73
4.2 Examples and practice . . . 73
4.3 Completion checklist . . . 73

5 Lecture 5: Corners, tracking, and matching 74
5.1 Planned scope . . . 74
5.2 Examples and practice . . . 74
5.3 Completion checklist . . . 74

6 Lecture 6: Structure from motion 75
6.1 Planned scope . . . 75
6.2 Examples and practice . . . 75
6.3 Completion checklist . . . 75

Study and reference 77

Formula reference 77
Lecture 1 . . . 77
Lecture 2 . . . 77

Quiz practice index 78
Lecture 1 . . . 78
Lecture 2 . . . 78
Lecture 3 . . . 79
Lecture 4 . . . 79
Lecture 5 . . . 80
Lecture 6 . . . 81
Outside the core scope . . . 81

Glossary 82

Sources and figure credits 86
Reading list named in the lecture . . . 86
Lecture 2 . . . 86

Errata and assumptions to verify 87
Lecture 1: verified qualifications and convention translations . . . 87
Lecture 2: verified corrections and assumptions . . . 88

Index 89

From images to geometry

How can photographs reveal the shape of a scene and the motion of a camera? This book develops the geometric ideas behind that question through the first six CS 532 lectures.

Read in lecture order, beginning with the *notation guide* and *prerequisite refresher*. Worked examples will connect the lecture slides to the course quizzes. The *quiz index* shows the planned placements.

> **Note**
>
> Lectures 1 and 2 now contain explanations, diagrams, worked quiz examples, and practice. Lectures 3–6 remain clearly labeled planning outlines. Material is added and reviewed one lecture at a time.

How to study a section

1. Start with the visual question and explain the intuition in your own words.
2. Read the formula's assumptions and symbol definitions before following the derivation.
3. Work through the application procedure, then attempt the example without its solution.
4. Use the chapter checklist and formula reference to identify what still needs practice.

The website and printable edition are built from the same Markdown pages. Essential explanations and solutions belong in both editions.

Before you begin

NOTATION AND COORDINATE CONVENTIONS

These conventions apply throughout the book. A source using another convention will be translated explicitly.

Symbol	Meaning	Shape / units
$\mathbf{X}$	Homogeneous 3D point	4×1; scale arbitrary
$\mathbf{x}$	Homogeneous image point	3×1; scale arbitrary
(u, v)	Inhomogeneous pixel coordinates	pixels
$\mathbf{X}_c$	Euclidean camera point $(X_c, Y_c, Z_c)^T$	3×1; one consistent length unit
$\mathbf{K}$	Camera intrinsic matrix	3×3; focal entries in pixels
$\mathbf{R}, \mathbf{t}$	World-to-camera rotation and translation	3×3, 3×1
$\mathbf{C}$	Camera center in world coordinates	3×1; $\mathbf{C} = -\mathbf{R}^T\mathbf{t}$
$\mathbf{P}$	Homogeneous camera projection matrix	3×4
$\mathbf{H}, \mathbf{F}, \mathbf{E}$	Homography, fundamental matrix, essential matrix	each 3×3, with different constraints
ℓ, π	Homogeneous line / plane coefficients	3×1 / 4×1

Frames, axes, and transformations

Vectors are columns. Image u increases rightward and v downward. Use right-handed camera axes with positive Z_c forward. Specify any different image-plane convention in the figure and derivation.

Write $\mathbf{T}_{B \leftarrow A}$ for a transform that maps coordinates from frame A into frame B. In a product, the rightmost transform acts first. State whether a rotation acts on points or changes the coordinate frame. Use scalar-first unit quaternions (q_w, q_x, q_y, q_z) and define the rotation convention before applying them.

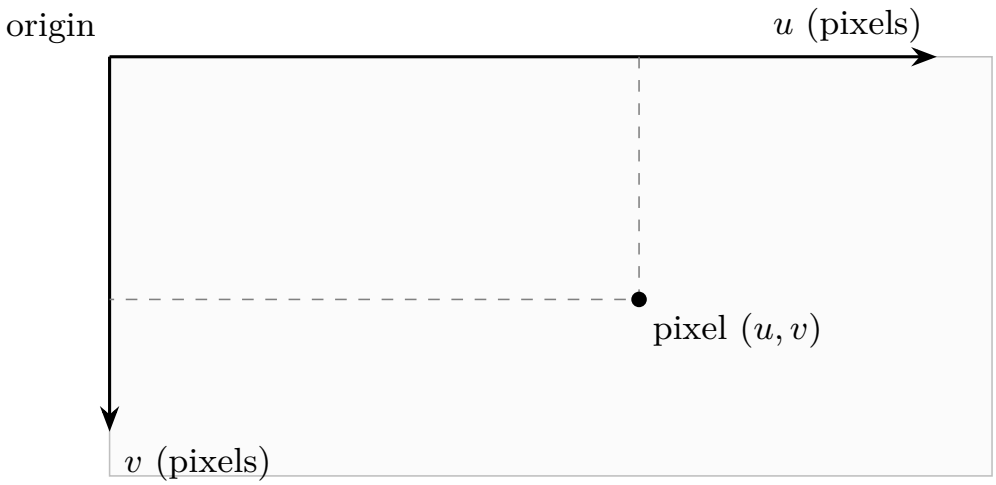


Image coordinates increase rightward and downward.
Distances are measured in pixels.

Fig. 1: The pixel-axis convention used throughout the book. The marked pixel is illustrative, not a calibrated measurement. Original diagram; editable TikZ source is retained with the figure assets.

Equality and homogeneous scale

Homogeneous coordinates represent a point by a nonzero vector up to nonzero scale; see *homogeneous coordinates*. Use $=$ for equality of vectors and $\sim$ for equality up to scale. A zero vector is never a projective point.

Formula — homogeneous scale equivalence

$$\mathbf{x} \sim \lambda \mathbf{x}, \qquad \lambda \neq 0. \tag{1}$$

Meaning: scaling all coordinates by the same nonzero number preserves the represented point. Here $\mathbf{x} \in \mathbb{R}^3 \setminus \{\mathbf{0}\}$ and λ is a scalar. This is an exact projective equivalence, not a numerical approximation. When the third coordinate w is nonzero, the corresponding Euclidean point is $(x_1/w, x_2/w)$. When $w = 0$, the point is ideal and this division is invalid.

Application — compare two representations

1. Check that neither vector is zero.
2. Determine whether one nonzero scalar multiplies every component of the first vector to produce the second.
3. For finite points, divide by the last coordinate and compare the resulting Euclidean coordinates.

For example, $(2, 4, 2)^T$ and $(1, 2, 1)^T$ represent the same finite point $(1, 2)$, although the two vectors are not equal. This is an original notation example; lecture-specific derivations and slide references will be added with the chapters.

Numerical checks

Keep pixels distinct from physical lengths. Convert centimeters and meters before substitution. Report exact values before rounded approximations. Distinguish a formula's mathematical conditions from numerical conditioning and state a justified tolerance when checking floating-point results.

You should be comfortable with algebra, trigonometry, basic probability, matrix operations, and introductory calculus. You do not need previous experience with 3D computer vision. Each lecture will refresh the specific tools it uses.

A quick readiness check

- **Vectors:** interpret a dot product, a cross product, and vector length geometrically.
- **Matrices:** check multiplication dimensions, distinguish an inverse from a transpose, and solve a small linear system.
- **Calculus:** interpret a derivative as local sensitivity and a first-order Taylor expansion as a local approximation.
- **Linear algebra:** distinguish eigenvectors from singular vectors and understand a nullspace as the set of solutions to a homogeneous linear system.
- **Probability:** calculate the complement of repeated independent failures.

Attempt the check below. Further numerical refreshers will be added when the corresponding lectures are written.

Check — can these matrices be multiplied?

A matrix A has shape 3×4 and a vector X has shape 4×1. What is the shape of AX? Can XA be formed?

Solution

The inner dimensions of AX agree, so its shape is 3×1. The product XA cannot be formed: its inner dimensions would be 1 and 3. This is the dimension check used later in camera projection.

Matrix and vector tools used in Lecture 1

A **basis** is a set of independent directions that can build every vector in the space. In 2D, $(1, 0)$ and $(0, 1)$ are the standard basis. The columns of a transformation matrix tell you where these basis vectors go. This is the derivation behind the rotation matrix in *Move points without losing the geometry*.

A **dot product** multiplies matching entries and adds them. For perpendicular vectors it is zero. In geometry it can test whether a point satisfies line or plane coefficients; the roles of the input vectors matter. A **cross product** of two 3-vectors is perpendicular to both, with magnitude equal to the area of their spanned parallelogram. Parallel inputs give zero. See (1.2) for its components and a geometric application.

The **transpose** swaps rows and columns; it does not generally invert a matrix. An **inverse** undoes the transformation and exists only for an invertible square matrix. Rotation matrices are special: their inverse equals their

transpose. A **determinant** is a signed area/volume scale for a square linear map. Zero means collapse of at least one independent direction; a proper rotation has determinant $+1$.

Rank, nullspaces, and SVD

Rank counts independent output directions. If a matrix sends a nonzero vector to zero, that vector belongs to its **nullspace**. A 3×4 matrix of rank three has a one-dimensional nullspace: every solution is a scalar multiple of one nonzero vector. This is precisely the scale ambiguity of a homogeneous plane or point.

A **singular value decomposition (SVD)** writes a real matrix as $A = U\Sigma V^T$, where U, V have orthonormal columns in their full forms and Σ contains nonnegative singular values. Geometrically, the map changes orthonormal coordinates, scales directions, then changes coordinates again. A right singular vector belonging to zero singular value is mapped to zero.

For the 3×4 incidence systems in *Points, planes, and lines in 3D*, a full SVD supplies a 4×4 V: its last column spans the nullspace when rank is three. A reduced decomposition may omit that column. This implementation detail matters even when the geometric derivation is correct.

An **eigenvector** satisfies $Av = \lambda v$: its direction is preserved, though its scale may change. A rotation axis has eigenvalue one. Eigenvectors and singular vectors solve different questions and should not be interchanged.

Refresher — a null vector is not the zero answer

For $A = \begin{pmatrix} 1 & 0 & -2 \\ 0 & 1 & -3 \end{pmatrix}$, verify a nonzero null vector and explain what "up to scale" means.

Solution

$A(2, 3, 1)^T = (0, 0)^T$. Every nonzero multiple of $(2, 3, 1)^T$ is another valid null vector. The zero vector is also an algebraic solution, but it cannot represent a projective point. The matrix has rank two and one free scale direction.

Part I

Lectures 1–6

CHAPTER

ONE

LECTURE 1: FOUNDATIONS AND 3D GEOMETRY

Can photographs taken from different viewpoints reveal a shared 3D world? Lecture 1 introduces that problem and builds the geometric language used to solve it.

This chapter follows the actual supplied deck, including its early introduction to camera projection. Later lectures revisit these foundations and develop their estimation algorithms. Read the sections in order; use the review checklist to find a formula or the final TLDR for a quick recap and quiz.

What you will learn

- Explain why a pixel alone does not determine scene depth.
- Join points, intersect lines, and recognize ideal points in homogeneous coordinates.
- Distinguish point transformations from line/plane transformations.
- Construct planes and spatial lines, and recognize degenerate inputs.
- Apply and compare rotation matrices, ordered angles, axis–angle, and quaternions.
- Compose rigid frame transforms with explicit directions.
- Project a world point into a camera and reason about camera motion.

Reading route

1. *Why reconstruct a world from photographs?*
2. *Points, lines, and infinity*
3. *2D transformations and invariants*
4. *Planes and lines in 3D*
5. *Rotation representations*
6. *Frames and integrated quiz problems*
7. *Camera projection and the hands-on exercise*
8. *Foundation quiz practice*
9. *Review and source-coverage checklist*
10. *TLDR: key takeaways and a short quiz*

How the quizzes fit

Fall 2025 Quiz 1, questions 3–4, develops relative camera pose and a camera/object plane. Fall 2017 Quiz 9 supplies the tetrahedron normals/rotation problem and a supplementary depth-backprojection bridge. The supplied lecture mapping classifies both quiz sets as supplementary; the core geometric ideas are still prerequisites for the higher-priority camera and two-view topics.

Spring 2020 Quiz 1 is also solved here: its original Lecture 2 assignment is preserved in the quiz index, but its foundation topics are taught only in this chapter.

Source: `wk1_2_cs532s26_IntroToGeometry.pdf`, all 114 PDF pages audited. Repeated visual builds are consolidated. Course logistics are excluded from the mathematics; research applications are summarized rather than treated as extra algorithms. Geometry slide sections credit M. Pollefeys and D. Cappelleri; image-formation sections credit John Oliensis. All newly added numerical values and diagrams are labeled as original or adapted.

1.1 Why reconstruct a world from photographs?

Imagine collecting photographs of the same building from people who never met. They used different cameras, stood in different places, and photographed it at different times. Could those disconnected pictures become one 3D model?

That is the opening challenge of Lecture 1: **"the Internet as a camera."** Here the phrase describes a collection of viewpoints, not one physical camera or a claim that arbitrary pictures can always be reconstructed. The reconstruction must discover which images overlap, which pixels refer to the same scene features, and where the cameras were.

Lecture source: `wk1_2_cs532s26_IntroToGeometry.pdf`, PDF pages 2–61. Printed slide labels match PDF pages through page 24; from PDF page 25 onward labels are offset by one. The application descriptions below summarize the supplied lecture, not the current state of a research project.

1.1.1 Work backward from an image

Computer vision infers properties of a scene from measured images. Computer graphics usually starts with scene geometry, materials, lighting, and a viewpoint to produce an image. Vision faces the inverse question: given the image, what could have produced it?

This distinction matters because the inverse need not be unique. A small nearby object and a larger distant object may occupy the same pixels. Surface color and illumination can also trade off: a dark pixel might belong to a dark material or a shadow. The ambiguous figures and lighting illusions on PDF pages 30–42 are reminders that an image is evidence, not a complete description of the world.

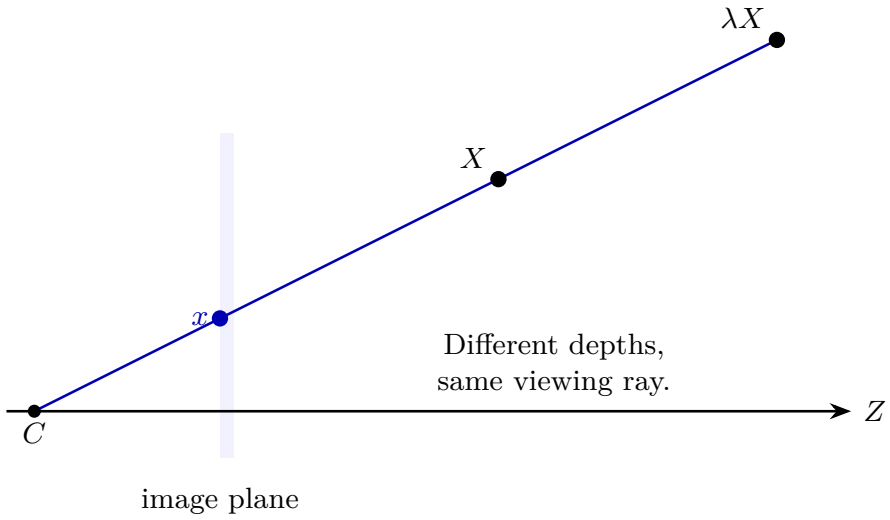


Fig. 1.1: One image point specifies a viewing ray, not a unique depth. Original diagram explaining the projection ambiguity illustrated on Lecture 1 PDF pages 37 and 105–107.

The picture gives us a useful distinction between **geometric information**—positions, shapes, and camera motion—and **semantic information**, such as whether a region depicts a building or vegetation. Both can help a vision system, but this book concentrates on geometry. A geometric reconstruction can have accurate surface positions without knowing the names of the objects.

1.1.2 A reconstruction is a chain of linked problems

The Colosseum sequence on PDF pages 19–24 motivates the following pipeline. The arrows are logical dependencies; practical systems may revisit earlier stages.

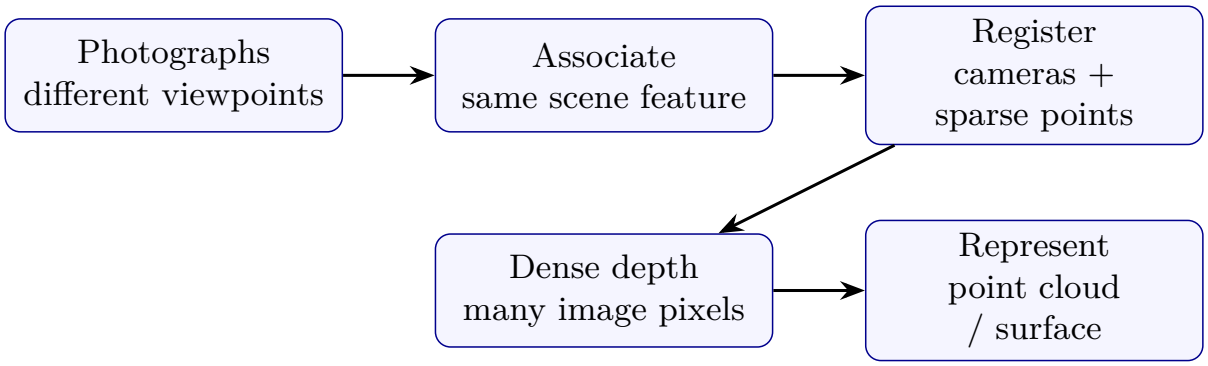


Fig. 1.2: From independent photographs to a geometric representation. Original redrawing of the lecture's reconstruction workflow, PDF pages 18–26. The underlying lecture figures use the Colosseum as an example.

1. **Associate:** identify measurements of the same scene feature across images. A similar-looking window is not automatically the same physical window.

2. **Register:** estimate camera positions/orientations and a sparse set of 3D points. **Structure from motion (SfM)** estimates scene structure and camera motion together from image correspondences.
3. **Estimate dense depth:** recover geometry for many pixels, not just a few distinctive corners.
4. **Represent:** organize the recovered geometry as a point cloud, surface, or another useful model. Dense depth, point-cloud fusion, and meshes are previews here; their full algorithms lie beyond these six lectures.

A ***depth map*** stores a depth value at each valid image location. A ***point cloud*** stores 3D point positions. They are different representations: a depth map still depends on its camera coordinates; a point cloud can express all points in one common frame.

1.1.3 Follow the fountain from pixels toward shape

PDF pages 50–51 show photographs, depth estimates, fusion, and colored geometry of a fountain. This is particularly useful because its decorative details are visible in both the photograph and geometric output.



Fig. 1.3: An input view from the fountain example, extracted from Lecture 1 PDF page 50, printed slide 51. Source: supplied Enrique Dunn lecture deck; the original photograph's creator is not identified on that slide.



Fig. 1.4: A depth output from the same example, extracted from PDF page 51, printed slide 52. The slide compares raw and fused estimates. Grayscale is an encoding of the source's depth data; its numerical scale is not specified, so do not infer metric depths from brightness alone. Source: supplied lecture deck; original figure creator not specified there.

Notice three things. First, matching must identify the same ornament or basin location in different views. Second, a depth estimate is a geometric hypothesis and can contain holes, noise, or mistakes near occlusion boundaries. Third, fusion must place estimates in a common reference frame; simply stacking pixel arrays would not align their 3D points.

The slides also show urban capture in Chapel Hill (PDF 47–49), large photo-collection reconstructions (52–54), visual SLAM (55), dynamic reconstruction (56), and remote sensing, autonomous systems, augmented reality, and geolocalization (57–61). These illustrate different data conditions, not extra algorithms to memorize in Lecture 1. **Visual SLAM** emphasizes estimating a moving camera's pose while building a map. A dynamic scene introduces an additional challenge: the same object may move between images.

1.1.4 Why the geometry is difficult

The lecture's examples identify several different obstacles:

- **Projection loses depth:** many 3D points can lie on one image ray.
- **Appearance is not geometry:** shadows, reflections, materials, and exposure change pixel intensities.
- **Measurements are imperfect:** noise, quantization, blur, occlusion, and camera-model errors affect the observations.
- **Association is ambiguous:** repeated textures and locally similar patches can produce incorrect matches.
- **More data creates more work:** a large photo collection can improve coverage but also increases matching and optimization costs.

The answer is not to abandon images. It is to state the model and assumptions clearly, use multiple sources of evidence, and check the resulting estimates.

Check 1.1 — is a correspondence already a 3D point?

Two images contain matching pixels on the same fountain ornament. Do those two pixel coordinates alone provide its position in meters?

Solution

No. We still need suitable camera geometry and a scale reference for metric units. Matching establishes data association. Reconstruction uses the image measurements and cameras to infer geometry. Even a valid monocular reconstruction generally has an unknown global scale unless additional information fixes it.

1.1.5 What to remember

The rest of Lecture 1 builds the language needed to describe this pipeline: homogeneous points and lines, transformations, planes, rotations, reference frames, and camera projection. The goal is to understand how a calculation relates to a geometric question, not to mistake a visually convincing output for a verified reconstruction.

1.2 Points, lines, and the meaning of infinity

Railway tracks are parallel in the world, yet their image traces appear to converge. Is that a contradiction? No: the world plane and the image plane are related by perspective, which does not preserve parallelism.

Lecture source: `wk1_2_cs532s26_IntroToGeometry.pdf`, PDF pages 64–71 (printed slides 65–72).



Fig. 1.5: The railway example from PDF page 68, printed slide 69. Source: supplied lecture deck; original photographer unspecified. The source image has limited resolution, so it is intentionally displayed at a modest size.

1.2.1 Add a coordinate, not an extra physical dimension

An ordinary point (u, v) can be represented by $\mathbf{x} = (u, v, 1)^T$. Multiplying all three components by a nonzero scalar gives the same projective point. The added coordinate makes certain geometric operations easier; it does not give a 2D point an extra physical dimension.

Recall from *Notation and coordinate conventions* that $\sim$ means equality up to nonzero scale. A finite point $(x_1, x_2, w)^T$ becomes $(x_1/w, x_2/w)$ when $we0$. For example, $(6, 9, 3)^T$ represents $(2, 3)$.

The ***projective plane*** $\mathbb{P}^2$ is the set of nonzero vectors in $\mathbb{R}^3$ **identified under scale**. This last clause matters: the slide's shorthand $\mathbb{R}^3 \setminus \{0\}$ must be understood with this identification. Three homogeneous entries have only two degrees of freedom because one overall scale is redundant.

Formula — point–line incidence

$$\ell^T \mathbf{x} = ax_1 + bx_2 + cw = 0. \tag{1.1}$$

Here $\ell = (a, b, c)^T \neq 0$ is a homogeneous line and $\mathbf{x} = (x_1, x_2, w)^T \neq 0$ is a point. For $w = 1$, this is the familiar $au + bv + c = 0$. The scalar zero tests ***incidence***: whether the point lies on the line.
Conditions and limits: This is exact. Both line and point coefficients may be rescaled. It tests incidence, not Euclidean distance; a nonzero residual has no scale-independent magnitude unless you normalize it. At least one of a, b must be nonzero for an ordinary finite line.

How to use it — point–line incidence

1. Express the point and line in the same coordinates.
2. Multiply corresponding entries and add them.
3. Check whether the result is zero (or numerically small under a specified normalization).

Worked connection: *Example 1.1*.

Why a cross product appears

The cross product of two 3-vectors is perpendicular to both. Therefore, if two vectors encode points, their cross product supplies line coefficients orthogonal to both points. If the vectors encode lines, their cross product supplies a point incident to both lines. The algebra is the same; the geometric interpretation changes.

Formula — join two points or intersect two lines

$$\ell \sim \mathbf{x} \times \mathbf{y}, \qquad \mathbf{p} \sim \ell_1 \times \ell_2,$$
$$\mathbf{a} \times \mathbf{b} = \begin{pmatrix} a_2 b_3 - a_3 b_2 \\ a_3 b_1 - a_1 b_3 \\ a_1 b_2 - a_2 b_1 \end{pmatrix}. \tag{1.2}$$

Each input and output is a 3×1 homogeneous vector. Joining two distinct points gives a line; intersecting two distinct lines gives a point. The components have coefficient-dependent units, so interpret the result after the appropriate homogeneous normalization.
Conditions and limits: The result must be nonzero. Equal points up to scale do not define a unique line; equal lines do not define a unique intersection. Reversing the order changes the sign and therefore preserves the represented object.

How to use it — join two points or intersect two lines

1. Decide whether the inputs represent points or lines.
2. Evaluate the three cross-product components.
3. If the result is zero, diagnose coincident/degenerate inputs.
4. For a point, divide by the last component only if it is nonzero. Verify incidence by dot products.

Worked connection: *Example 1.1*.

Worked example 1.1 — parallel lines have a projective intersection

Source: original numerical expansion of the railway/ideal-point example, PDF pages 67–71. The related Spring 2020 Quiz 1 is solved in *Foundation quiz practice*.

Take $y = 1$ and $y = 2$, represented by $\ell_1 = (0, 1, -1)^T$ and $\ell_2 = (0, 1, -2)^T$.

1. Their cross product is $(-1, 0, 0)^T \sim (1, 0, 0)^T$.
2. The last coordinate is zero, so this is an ***ideal point***, not a finite Euclidean position.
3. Check incidence: both $\ell_1^T(1, 0, 0)$ and $\ell_2^T(1, 0, 0)$ are zero.
4. Its first two coordinates identify the horizontal direction. Division by the last coordinate is neither required nor allowed.

For comparison, intersecting $x = 2$, or $(1, 0, -2)^T$, with $y = 3$, or $(0, 1, -3)^T$, gives $(2, 3, 1)^T$, a finite point.

Formula — ideal points and the line at infinity

$$\begin{aligned} \ell_1 \times \ell_2 &= (c_2 - c_1)(b, -a, 0)^T, \\ \ell_\infty &= (0, 0, 1)^T, \qquad \ell_\infty^T (d_x, d_y, 0)^T = 0. \end{aligned} \tag{1.3}$$

The parallel finite lines are $\ell_1 = (a, b, c_1)^T$ and $\ell_2 = (a, b, c_2)^T$. Their ideal intersection records direction $(b, -a)$. All points with last coordinate zero lie on the ***line at infinity***.

Conditions and limits: Require $(a, b) \neq (0, 0)$ and $c_1 \neq c_2$. The factor $c_2 - c_1$ may be dropped up to scale, but not when it is zero. "At infinity" is a projective classification, not a very large finite coordinate.

How to use it — ideal points and the line at infinity

1. Put distinct parallel lines into a shared (a, b) normalization.
2. Cross them and recognize the zero last component.
3. Read the direction from the first two components and check it is perpendicular to (a, b).

Worked connection: *Example 1.1*.

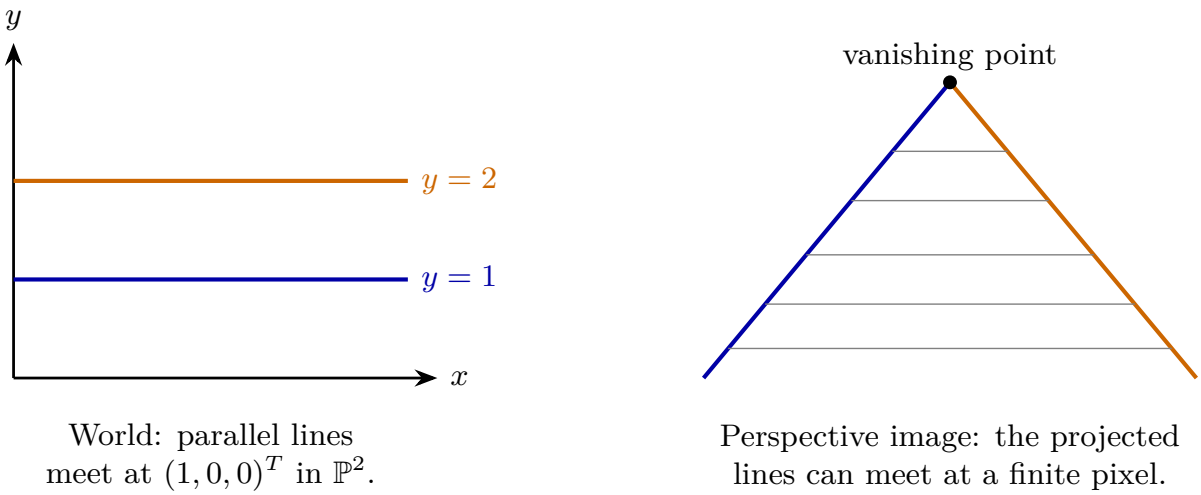


Fig. 1.6: An ideal point in one plane may map to a finite vanishing point in another. Original diagram developing PDF pages 68–71. The world and image panels are different coordinate spaces, not two claims about the same pair of coordinate lines.

1.2.2 A vanishing point is not automatically an ideal image point

A scene's parallel rails share a direction. Under a perspective mapping, that direction can project to a finite pixel location: the ***vanishing point*** in the photograph. The ***horizon*** of a scene plane is the image of that plane's line at infinity. It need not coincide with the image's own line at infinity.

This distinction explains the opening puzzle. The world lines remain parallel in world coordinates, while their images can intersect at an ordinary finite pixel. We will derive the camera mapping later in *Image formation: where does a 3D point appear?*.

Check 1.2 — does changing scale change the line?
Do $(2, -2, 4)^T$ and $(-1, 1, -2)^T$ describe different lines? Does $(0, 0, 0)^T$ describe every line?

Solution

The first two vectors differ by a factor of -2, so they describe the same line. The zero vector is invalid: its equation places no restriction on points and cannot identify a unique line.

1.2.3 What to remember

Use dot products for incidence and cross products for joins/intersections. Normalize a finite point only after checking its last coordinate. Track which space a point belongs to before calling it a vanishing point or a point at infinity.

1.3 Move points without losing the geometry

Suppose a point is one unit to the right of a pivot. A quarter-turn should put it one unit above that pivot. Rotating around the coordinate origin instead gives the wrong answer unless the pivot happens to be the origin. A small coordinate diagram makes this distinction easier than memorizing a large matrix.

Lecture source: `wk1_2_cs532s26_IntroToGeometry.pdf`, PDF pages 72–79 (printed slides 73–80).

Throughout the 2D rotation examples, x points right and y points **up** in a Cartesian diagram. This differs from the downward pixel v axis in the notation guide. We use active rotations of points and the right-hand rule; do not transfer a screen-clockwise interpretation without checking the axes.

1.3.1 Derive rotation from the basis vectors

A counterclockwise rotation by $heta$ sends $(1,0)^T$ to $(\cos heta, \sin heta)^T$. It sends $(0,1)^T$ to $(-\sin heta, \cos heta)^T$. A point $(x,y)^T$ is x times the first basis vector plus y times the second. Rotating both basis vectors and combining them gives the columns of the rotation matrix.

Formula — 2D rotation

$$\mathbf{p}' = R(\theta)\mathbf{p}, \qquad R(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}. \tag{1.4}$$

$\mathbf{p}$ and $\mathbf{p}'$ are 2×1 positions relative to the rotation center; R is 2×2 and unitless. Thus $x' = x\cos\theta - y\sin\theta$ and $y' = x\sin\theta + y\cos\theta$.
Conditions and limits: Exact for a planar rotation. State angle units before evaluating trigonometric functions. This rotates around the origin and preserves lengths and angles; translation is a separate operation.

How to use it — 2D rotation

1. Choose a Cartesian axis convention and rotation center.
2. Convert the angle to the units expected by the computation.
3. Multiply the matrix by the point; check that its distance from the center is unchanged.

Worked connection: *Example 1.2*.

Formula — translation and rotation about a pivot

$$\begin{pmatrix} \mathbf{p}' \\ 1 \end{pmatrix} = \begin{pmatrix} R & \mathbf{t} \\ 0 & 1 \end{pmatrix} \begin{pmatrix} \mathbf{p} \\ 1 \end{pmatrix}, \qquad \mathbf{p}' = \mathbf{c} + R(\mathbf{p} - \mathbf{c}), \qquad H_c = T(\mathbf{c})\,\mathrm{diag}(R, 1)T(-\mathbf{c}). \tag{1.5}$$

$\mathbf{p}, \mathbf{c}, \mathbf{t}$ are 2×1 vectors in the same length unit. $T(\mathbf{t})$ is the 3×3 homogeneous translation matrix with identity upper-left block and last column $(t_x, t_y, 1)^T$.
Conditions and limits: Exact rigid geometry; the rightmost matrix acts first. A direction has homogeneous last coordinate zero, so a translation does not change it. A position has last coordinate one.

How to use it — translation and rotation about a pivot

1. Subtract the pivot to express the point relative to it.
2. Apply the rotation.
3. Add the pivot back. In homogeneous form multiply in that same right-to-left order.
4. Check that the pivot itself stays fixed.

Worked connection: *Example 1.2*.

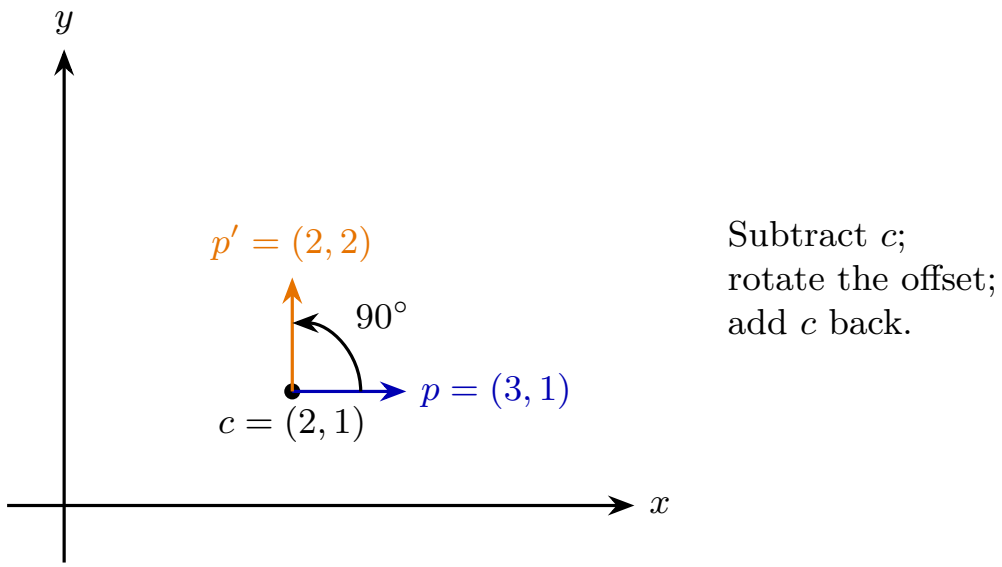


Fig. 1.7: Translate to the pivot, rotate the offset, then translate back. Original solution diagram for the hands-on question on PDF page 77, printed slide 78.

Worked example 1.2 — a quarter-turn around a chosen center

Source: numerical adaptation of the hands-on tasks on PDF page 77.
Let $p = (3, 1)^T$, $c = (2, 1)^T$, and $\theta = 90°$.

1. The offset is $p - c = (1, 0)^T$.
2. A quarter-turn sends that offset to $(0, 1)^T$.

3. Adding the center gives $p' = (2, 2)^T$.
4. Both offsets have length one, and applying the same operation to c gives c.

Rotating p directly around the origin would give $(-1, 3)^T$, demonstrating why the chosen center matters.

1.3.2 Transformation families: what stays unchanged?

The **degrees of freedom (DOF)** count independent parameters, not the number of printed matrix entries. A general invertible 3×3 homogeneous matrix has nine entries, but its overall nonzero scale is irrelevant, leaving eight DOF.

Formula — 2D transformation hierarchy

$$H_E = \begin{pmatrix} R & t \\ 0^T & 1 \end{pmatrix}, \quad H_S = \begin{pmatrix} sR & t \\ 0^T & 1 \end{pmatrix}, \quad H_A = \begin{pmatrix} A & t \\ 0^T & 1 \end{pmatrix}, \quad H_P = \begin{pmatrix} A & t \\ v^T & \alpha \end{pmatrix}. \tag{1.6}$$

Blocks R, A are 2×2; t, v are 2×1 and α, s are scalars. Euclidean, similarity, affine, and projective maps have respectively 3, 4, 6, and 8 DOF in 2D. Here $s > 0$ and R is a proper rotation.

Conditions and limits: Each full mapping must be invertible; affine A must be invertible. Projective division is required after multiplication and can map finite points to infinity. The Euclidean form here excludes reflections; allowing reflections adds a discrete choice, not another continuous DOF.

How to use it — 2D transformation hierarchy

1. Ask what the data require: rigid motion, uniform scale, shear, or perspective.
2. Identify the smallest valid transformation family.
3. Check its invariants using the table below and normalize transformed finite points.

Worked connection: *Example 1.3*.

Family	What it preserves	What it can change
Euclidean	Lengths, angles, incidence, parallelism	Position and orientation
Similarity	Angles and ratios of lengths	Overall scale
Affine	Incidence, parallelism, ratios along a line, ratios of areas	Angles and unequal directional scales
Projective	Incidence, straightness, cross-ratio where defined	Parallelism, lengths, angles, ordinary area ratios

The table summarizes PDF page 78. "Preserves ratios" for an affine map means the relevant ratio within the same transformation, not that every arbitrary pair of vector lengths is preserved. A projectivity also preserves tangency where defined; it need not preserve a visible square's appearance.

Worked example 1.3 — shear is not a rotation

Source: original computation developing the transformation hierarchy on PDF page 78.

Take $A = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$. It sends the basis vectors to $(1, 0)^T$ and $(1, 1)^T$.

1. Their original dot product is zero; their new dot product is one. The right angle changed.
2. Horizontal parallel lines remain horizontal and parallel.
3. The determinant is one, so oriented area is unchanged for this particular shear.

This is affine, but not Euclidean or similarity. General affine maps multiply every area by the same determinant magnitude, which explains why area ratios are preserved even when areas themselves change.

The hierarchy slide also names the ***circular points***: two complex points at infinity shared by all Euclidean circles in a plane. Their invariance under similarities is a projective way of encoding angle information; they are not finite pixels you could photograph. In 3D, the analogous ***absolute conic*** is a complex conic on the plane at infinity that encodes the Euclidean metric. These are vocabulary previews for reconstruction upgrades in Lecture 6, not additional numerical tasks for Lecture 1.

1.3.3 Lines transform differently from points

A line is a set of points satisfying $\ell^T x = 0$. If $x' = Hx$, then $x = H^{-1}x'$. Substituting gives $\ell^T H^{-1} x' = 0$. Transpose the coefficient product: $(H^{-T}\ell)^T x' = 0$. That is why multiplying a line by H generally gives the wrong result.

Formula — transform line and plane coefficients

$$\mathbf{x}' \sim H\mathbf{x}, \qquad \ell' \sim H^{-T}\ell; \qquad \mathbf{X}' \sim G\mathbf{X}, \quad \pi' \sim G^{-T}\pi. \tag{1.7}$$

H is an invertible 3×3 planar projectivity; G is an invertible 4×4 spatial projectivity. Lines and planes are **dual** coefficients of incidence equations, so they transform with the inverse transpose.

Conditions and limits: Exact projective incidence. The inverse does not exist for a singular transformation; such a map is not a projectivity. Coefficient normalization does not change the represented line or plane.

How to use it — transform line and plane coefficients

1. Confirm whether the object is a point/direction or an incidence coefficient vector.
2. Use the forward matrix for points and its inverse transpose for lines/planes.
3. Transform a known incident point and verify that its new incidence residual is zero.

Worked connection: *Example 1.4*.

Worked example 1.4 — translate a line

Source: original verification of PDF page 79's line-transformation rule.

Translate every point by $(2, 0)^T$. The line $x = 0$ should become $x' = 2$.

Its coefficients are $\ell = (1, 0, 0)^T$. With $H = \begin{pmatrix} 1 & 0 & 2 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$, the correct result is $H^{-T}\ell = (1, 0, -2)^T$, or $x' - 2 = 0$. Applying H directly to ℓ would leave the line at $x' = 0$, which fails the geometric check.

Check 1.3 — reverse the operation

After the pivot rotation in Example 1.2, how do you recover the original point?

Solution

Subtract the same pivot, rotate by $-90°$, then add the pivot back. The inverse reverses the operation order: $(ABC)^{-1} = C^{-1}B^{-1}A^{-1}$. For this pivot sandwich the result is $T(c)R(-90°)T(-c)$.

1.4 Points, planes, and lines in 3D

A 2D line is described by one linear equation. In 3D, one linear equation describes a plane instead. A spatial line therefore needs more structure: it is the intersection of two planes, or a point together with a direction.

Lecture source: `wk1_2_cs532s26_IntroToGeometry.pdf`, PDF pages 80–87, printed slides 81–88.

1.4.1 Homogeneous 3D points

Represent a finite point (X, Y, Z) by $(X, Y, Z, 1)^T$. Divide a general homogeneous point $(X_1, X_2, X_3, X_4)^T$ by X_4 when $X_4 \neq 0$. When $X_4 = 0$, it represents an ideal direction. The zero vector is invalid.

An invertible 4×4 matrix G acts by $\mathbf{X}' \sim G\mathbf{X}$. It has $16 - 1 = 15$ projective DOF because one overall scale is redundant. A plane transforms by G^{-T}; the incidence derivation in *Move points without losing the geometry* applies without change.

1.4.2 A plane is a normal and an offset

A ***surface normal*** is perpendicular to the surface's local tangent directions. For a plane it is constant everywhere. If $n = (a, b, c)^T$, the equation $n^T q + d = 0$ describes points whose displacement from a point on the plane is perpendicular to n.

Formula — plane incidence and Euclidean distance

$$\pi^T X = 0, \quad \pi = (a, b, c, d)^T, \qquad D(q, \pi) = \frac{|n^T q + d|}{\|n\|}. \tag{1.8}$$

$X = (q^T, 1)^T$ is 4×1; q, n are 3×1. D is unsigned point-to-plane distance in the coordinate length unit. The signed version omits the absolute value and depends on normal orientation.

Conditions: $n \neq 0$ for a finite Euclidean plane. The plane at infinity $(0, 0, 0, 1)^T$ is projectively valid but has no finite distance formula. Exact geometry; normal scale is arbitrary.

How to use it — test incidence or measure distance

1. Put the point and plane in the same frame.
2. Evaluate $n^T q + d$; zero means incidence.
3. Divide its magnitude by $\|n\|$ for distance.
4. Check that rescaling all four coefficients leaves the distance unchanged.

See Example 1.5 below.

Formula — plane through three noncollinear points

$$n = (B - A) \times (D - A), \qquad d = -n^T A, \qquad \pi = (n^T, d)^T. \tag{1.9}$$

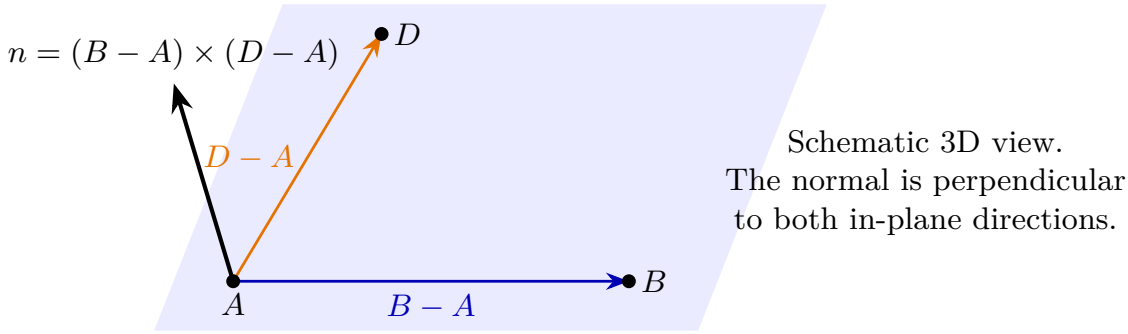
A, B, D are 3×1 Euclidean positions. Their differences lie in the plane, so their cross product is perpendicular to it. Substituting A fixes d.

Conditions: the points must be noncollinear. A zero cross product means no unique plane. Reversing its order reverses the normal but preserves the unoriented plane. If coordinates have length units, the unnormalized normal has squared-length units; normalize before metric interpretation.

How to use it — construct and verify a plane

1. Subtract the same anchor point to form two directions.
2. Cross them and check for a nonzero result.
3. Compute d and substitute all three points into the plane equation.
4. Normalize n only when a unit normal or signed metric distance is needed.

See *Example 1.5* and the camera/object quiz in *Reference frames and two worked quiz problems*.



Schematic 3D view. The normal is perpendicular to both in-plane directions.

Fig. 1.8: A normal comes from two in-plane directions. Original schematic developing PDF pages 82–84. Apparent angles in this drawing are not measurements of the 3D angles.

Worked example 1.5 — find a plane, then check a distance

Source: original numerical development of PDF pages 82–84.
Let $A = (1, 0, 0)$, $B = (0, 1, 0)$, and $D = (0, 0, 1)$.

1. $B - A = (-1, 1, 0)$ and $D - A = (-1, 0, 1)$.
2. Their cross product is $(1, 1, 1)$, so $d = -1$.
3. The plane is $x + y + z - 1 = 0$. All three input points satisfy it.
4. The origin's distance is $1/\sqrt{3}$ units, not one: the normal is not unit length.

1.4.3 The same geometry from a nullspace

A ***nullspace*** consists of vectors a matrix sends to zero. Stacking homogeneous input points as rows makes a plane coefficient vector orthogonal to every row. Conversely, stacking planes gives the point incident to all of them.

Formula — point/plane incidence systems

$$\underbrace{\begin{pmatrix} X_A^T \\ X_B^T \\ X_D^T \end{pmatrix}}_{M\,:\,3\times4} \pi = 0, \qquad \underbrace{\begin{pmatrix} \pi_1^T \\ \pi_2^T \\ \pi_3^T \end{pmatrix}}_{N\,:\,3\times4} X = 0. \tag{1.10}$$

Each unknown is a nonzero 4×1 vector, unique up to scale when the coefficient matrix has rank three. An SVD $M = U\Sigma V^T$ supplies the null vector as the last column of the full 4×4 matrix V.
Conditions: exact rank-three incidence. Lower rank means nonuniqueness. A reduced SVD of a 3×4 matrix can omit the null vector: request the full right-singular basis. An intersection with last coordinate zero is ideal, not finite.

How to use it — solve homogeneous incidence

1. Put point or plane coefficients in rows.
2. Check rank and solve for a nonzero right-null vector.
3. Normalize only when valid; substitute back into every equation.
4. For numerical data state a rank tolerance and inspect the residual.

See *Example 1.6*.

The determinant construction on PDF page 84 is another way to express the same plane. Four coplanar homogeneous point columns are linearly dependent, so their 4×4 determinant vanishes. Expanding along the variable point column yields the plane coefficients.

Formula — plane coefficients by cofactors

$$\det\begin{pmatrix} X & X_A & X_B & X_D \end{pmatrix} = 0, \qquad \pi = (D_{234}, -D_{134}, D_{124}, -D_{123})^T. \tag{1.11}$$

D_{ijk} is the 3×3 determinant formed from rows i, j, k of the last three columns. Expansion gives $X_1 D_{234} -$

$X_2 D_{134} + X_3 D_{124} - X_4 D_{123} = 0$.
Conditions: independent input point columns. This exact construction equals the nullspace result; near degeneracy can make direct cofactor evaluation poorly conditioned.

How to use it — evaluate plane cofactors

1. Form the 4×3 matrix of the input point columns.
2. Delete each row and calculate the remaining determinant.
3. Apply alternating signs and verify incidence at each input point.

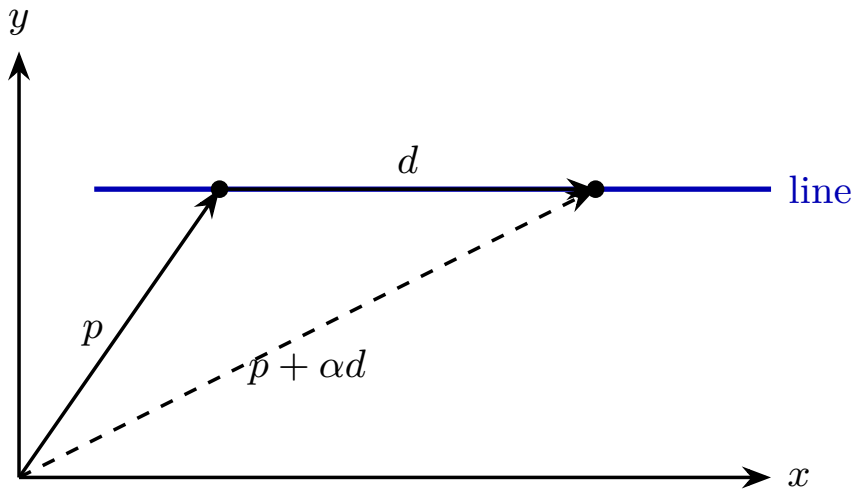
See *Example 1.6*.

Worked example 1.6 — duality in numbers

For Example 1.5 the point rows are $(1, 0, 0, 1)$, $(0, 1, 0, 1)$, and $(0, 0, 1, 1)$. Multiplying by $(1, 1, 1, -1)^T$ gives zero. The cofactors are $1, -1, 1, 1$ in the order above, giving the same plane after alternating signs. Conversely, the planes $x = 1$, $y = 2$, and $z = 3$ have rows $(1, 0, 0, -1)$, $(0, 1, 0, -2)$, and $(0, 0, 1, -3)$. Their null vector is $(1, 2, 3, 1)^T$, so their intersection is $(1, 2, 3)$.
Source: original numerical expansions of PDF pages 83–85.

1.4.4 Plücker coordinates: a line is a direction and a moment

Plücker coordinates encode a nonzero direction d and a moment $m = p \times d$, where p is any point on the line. Choosing a different point along the line should not change its representation.



Changing the chosen point along the line does not change $m = p \times d$, because $d \times d = 0$.

Fig. 1.9: The chosen point may move along the line without changing its moment. Original diagram developing PDF pages 86–87.

Formula — finite-line Plücker coordinates

$$L = (d, m), \qquad m = p \times d, \qquad d^T m = 0, \qquad p_\perp = \frac{d \times m}{\|d\|^2}. \tag{1.12}$$

All component vectors are 3×1. $p_\perp$ is the closest point on the line to the origin. With unitless $d, p, m, p_\perp$ have length units. A common nonzero scale on (d, m) represents the same line.
Conditions: $d \neq 0$. Six entries minus one scale and one orthogonality constraint leave four line DOF. Not every six-vector is valid. Lines entirely at infinity require a more general treatment than this finite-line form.

How to use it — encode or recover a spatial line

1. Choose a point and a nonzero direction; calculate $m = p \times d$.
2. Check $d^T m = 0$.
3. Recover $p_\perp$ if useful and check it belongs to the same line.
4. Keep the direction/moment order and cross-product sign consistent.

See *Example 1.7*.

Replacing p by $p + \alpha d$ leaves the moment unchanged because $d \times d = 0$. The closest-point formula follows from

$$d \times (p \times d) = \|d\|^2 p - (d^T p) d.$$

After division by $\|d\|^2$, the component along the line is removed, leaving the perpendicular displacement from the origin.

Formula — reciprocal product and line distance

$$\begin{aligned} \rho &= d_1^T m_2 + d_2^T m_1, \\ D &= \frac{|\rho|}{\|d_1 \times d_2\|} \quad (d_1 \times d_2 \neq 0), \\ D_{\parallel} &= \frac{\|(p_2 - p_1) \times d_1\|}{\|d_1\|}. \end{aligned} \tag{1.13}$$

$L_i = (d_i, m_i)$ denotes each valid finite line; p_i is any point on it. The ***reciprocal product*** ρ is zero for coplanar lines, including both intersecting and parallel lines. D is their shortest distance in the coordinate length unit.
Conditions: nonzero directions. Use the parallel expression when directions are parallel; the nonparallel formula is unstable near parallelism. A zero reciprocal product alone does not distinguish intersection from parallelism.

How to use it — measure separation

1. Check the direction cross product to classify the case.
2. For nonparallel lines, divide the reciprocal-product magnitude by the cross-product norm.
3. For parallel lines, measure the perpendicular part of the displacement between two points.
4. Check nonnegativity, units, and invariance under common rescaling of a line's coordinates.

See *Example 1.7*.

The nonparallel formula measures displacement along a common perpendicular. Substituting $m_i = p_i \times d_i$ gives $\rho = (p_1 - p_2)^T (d_1 \times d_2)$. The denominator turns the common perpendicular into a unit vector.

Worked example 1.7 — two skew lines are two units apart

Source: original numerical companion to PDF page 87; supplementary relative to the exam priorities.
Let $p_1 = (0, 1, 0)$, $d_1 = (1, 0, 0)$, $p_2 = (0, 0, 2)$, and $d_2 = (0, 1, 0)$.

1. $m_1 = (0, 0, -1)$ and $m_2 = (-2, 0, 0)$; each satisfies $d_i^T m_i = 0$.
2. $d_1 \times d_2 = (0, 0, 1)$ has norm one.
3. $\rho = -2$, so $D = 2$.
4. The first line lies at $z = 0$ and the second at $z = 2$. Their closest points have equal x, y, confirming the result.

Check 1.4 — does any triplet determine a plane?

What happens for $(0, 0, 0)$, $(1, 0, 0)$, and $(2, 0, 0)$?

Solution

The points are collinear. Their displacement cross product is zero, so infinitely many planes contain the common line. Do not normalize the zero vector or claim a unique solution.

1.5 Rotations: several descriptions of the same motion

A 3D orientation can be stored as a matrix, an axis and angle, three ordered angles, or a quaternion. These representations are not interchangeable lists of numbers. Ask **what rotates, about which axes, and in which order?** before using any of them.

Lecture source: `wk1_2_cs532s26_IntroToGeometry.pdf`, PDF pages 88–100, printed slides 89–101.

1.5.1 Rotation matrices preserve the dot product

We use active right-handed rotations of column vectors. Each elementary rotation keeps its named axis fixed and performs a 2D rotation in the perpendicular plane.

Formula — elementary 3D rotations

$$R_x(\theta) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & c & -s \\ 0 & s & c \end{pmatrix}, \quad R_y(\theta) = \begin{pmatrix} c & 0 & s \\ 0 & 1 & 0 \\ -s & 0 & c \end{pmatrix}, \quad R_z(\theta) = \begin{pmatrix} c & -s & 0 \\ s & c & 0 \\ 0 & 0 & 1 \end{pmatrix}, \qquad c = \cos\theta, \quad s = \sin\theta. \tag{1.14}$$

Each matrix is unitless and 3×3; the point it multiplies is 3×1. Angles follow the right-hand rule in a right-handed Cartesian frame.
Conditions: exact rotations about axes through the origin. A displaced axis needs translations as well. A passive change of coordinates uses the inverse. Specify degrees or radians explicitly.

How to use it — rotate around a named axis

1. Name the axis and state whether points or coordinates are rotating.
2. Substitute sine/cosine in consistent angle units.
3. Multiply by the point.
4. Check the fixed coordinate and preserved length.

See *Example 1.8* and the tetrahedron quiz in *Reference frames and two worked quiz problems*.

Formula — validate and invert a rotation

$$R^T R = I, \qquad \det R = +1, \qquad R^{-1} = R^T. \tag{1.15}$$

The columns of a proper rotation are orthonormal basis vectors. Dot products are preserved because $(Ra)^T(Rb) = a^T(R^T R)b = a^T b$; therefore lengths and angles are preserved. Nine entries with six independent orthonormality constraints leave three continuous DOF.
Conditions: a proper rotation, not an arbitrary matrix. An orthogonal matrix with determinant -1 includes a reflection. Floating-point checks need tolerances; transposing a noisy nonrotation is not generally its inverse.

How to use it — check a claimed rotation

1. Compare $R^T R$ with identity and $\det R$ with $+1$.
2. Use R^T as the inverse only when those conditions hold.
3. Rotate a test vector forward and backward and compare with the original.

See *Example 1.8*.

The slide example on PDF page 89 sends $(1,0,1)^T$ to $(0,1,1)^T$ with a positive quarter-turn about z. The unchanged last coordinate and equal lengths give two independent sanity checks.

1.5.2 Composition: order is part of the answer

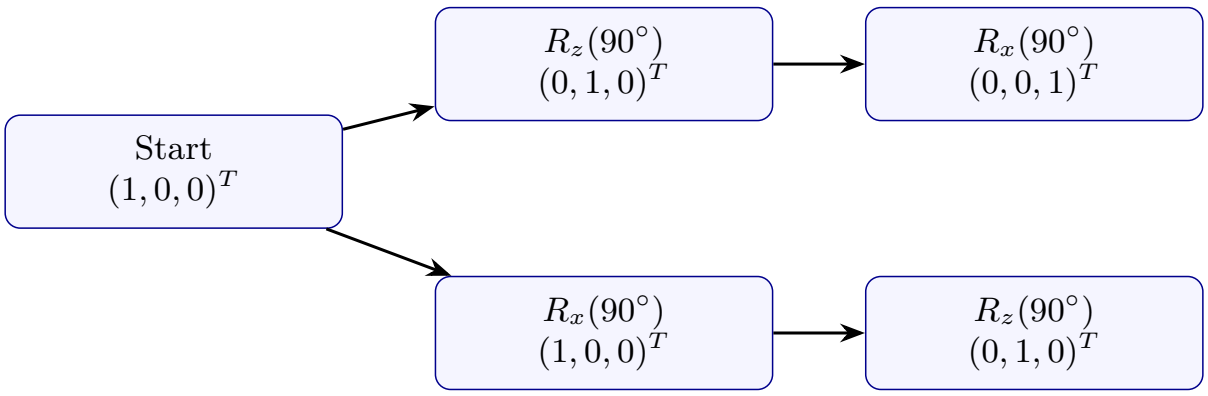


Fig. 1.10: The same two rotations in different orders give different results. Original numerical diagram developing PDF pages 93–100.

Worked example 1.8 — prove that rotation order matters

Source: original numerical expansion of the compositional-rotation slides, PDF pages 93–100.
Start with $v = (1,0,0)^T$.

1. Apply $R_z(90°)$: v becomes $(0,1,0)^T$.

2. Apply $R_x(90^\circ)$ next: it becomes $(0, 0, 1)^T$.
3. Reverse the order: the first x rotation leaves v unchanged, then the z rotation gives $(0, 1, 0)^T$.

Thus $R_x R_z \neq R_z R_x$. The rightmost matrix in each product acts first on the column vector.

1.5.3 Body axes versus fixed axes

An ***intrinsic rotation*** sequence uses axes attached to the moving body. An ***extrinsic rotation*** sequence uses fixed reference axes. Euler terminology varies: some sources reserve "Euler angles" for a repeated first/third axis, while others also include three-distinct-axis sequences. Write the axes and matrix product explicitly.

The lecture's body-axis sequence uses x, then the new y, then the new z (PDF 95–98). The fixed-axis sequence uses premultiplication (PDF 99–100). Equal angle values in these two sequences do not generally give equal orientations.

Formula — ordered-angle composition

$$R_{\text{body } xyz} = R_x(a)R_y(b)R_z(c), \qquad R_{\text{fixed } xyz} = R_z(c)R_y(b)R_x(a). \tag{1.16}$$

Angles a, b, c name successive x, y, z rotations. For an orientation matrix mapping body coordinates to world coordinates, a body-axis increment postmultiplies it; a world-axis increment premultiplies it. All factors are 3×3.

Conditions: the sequence and frame interpretation are essential. At $b = \pm 90^\circ$ these three-angle parameterizations become singular: two angle directions lose independent effect, called ***gimbal lock***. The physical orientation remains valid; its angle description becomes ambiguous.

How to use it — assemble an orientation

1. Specify body versus fixed axes and the full sequence.
2. Associate each angle with its named axis.
3. Multiply the elementary matrices in the stated order.
4. Test a basis vector instead of trusting roll/pitch/yaw names alone.

See *Example 1.8*.

For reference, multiplying the body-xyz factors gives

$$\begin{pmatrix} c_b c_c & -c_b s_c & s_b \\ s_a s_b c_c + c_a s_c & -s_a s_b s_c + c_a c_c & -s_a c_b \\ -c_a s_b c_c + s_a s_c & c_a s_b s_c + s_a c_c & c_a c_b \end{pmatrix},$$

with $c_a = \cos a$, $s_a = \sin a$, and similarly for b, c. This is an expansion of the boxed product, not a different rule. Recover it by multiplying two factors first and checking row–column dot products. Roll/pitch/yaw labels depend on axis convention; the lecture's fixed-axis figure uses its own assignment. An explicit product avoids that ambiguity.

1.5.4 Axis–angle: turn about one direction

An **axis–angle representation** uses a unit axis a and a turn angle θ. Components along the axis stay fixed. Components perpendicular to it undergo the familiar planar rotation.

Formula — extract axis and angle

$$\theta = \arccos\left(\frac{\operatorname{tr}(R) - 1}{2}\right), \qquad a = \frac{1}{2\sin\theta}\begin{pmatrix} R_{32} - R_{23} \\ R_{13} - R_{31} \\ R_{21} - R_{12} \end{pmatrix}. \tag{1.17}$$

R is a proper 3×3 rotation and $\theta \in [0, \pi]$ is its principal angle. a is a unit 3×1 direction. The trace is the diagonal sum: one unchanged axial component contributes one, while the perpendicular rotation contributes $2\cos\theta$.
Conditions: the axis expression requires $\sin\theta \neq 0$. At zero angle the axis is arbitrary. At π, recover the axis with a stable method, such as a unit eigenvector satisfying $Ra = a$. After validating R, clamp small numerical arccos overshoots to $[-1, 1]$.

How to use it — recover a geometric rotation

1. Validate R, calculate its trace, and recover the angle.
2. Handle near-zero and near-π cases separately.
3. Otherwise compute and normalize the axis.
4. Check $Ra = a$ and reconstruct a test rotation.

See *Example 1.9*.

An original derivation companion makes this representation computational. Split p into $p_{\parallel} = a(a^T p)$ and $p_{\perp} = p - p_{\parallel}$. Keep the first part, rotate the second by sine/cosine, and use $a \times p$ as its quarter-turn direction. Collecting terms yields **Rodrigues' formula**:

Formula — build a rotation from axis and angle

$$R = I\cos\theta + (1 - \cos\theta)aa^T + \sin\theta[a]_{\times}, \qquad [a]_{\times}p = a \times p. \tag{1.18}$$

Here $[a]_{\times} = \begin{pmatrix} 0 & -a_3 & a_2 \\ a_3 & 0 & -a_1 \\ -a_2 & a_1 & 0 \end{pmatrix}$. Every matrix is 3×3 and unitless.
Conditions: $\|a\| = 1$, active right-handed rotation, and consistent angle units. This exact formula is an explanatory supplement to PDF page 91; it is not claimed to be printed on that slide.

How to use it — construct a rotation matrix

1. Normalize the axis and evaluate sine/cosine.
2. Form the outer product aa^T and cross-product matrix.

3. Add the three terms.
4. Check the axis stays fixed and the result is a proper rotation.

See *Example 1.9*.

1.5.5 Quaternions: four entries for a three-DOF rotation

A ***quaternion*** is a scalar–vector pair $q = (w, v)$ with a special multiplication rule. We use scalar-first Hamilton multiplication and active rotations. Unit length removes one DOF from four entries. The pair $q, -q$ represents the same rotation; this is a discrete ambiguity.

Formula — unit quaternion and active point rotation

$$\begin{aligned} q &= \left(\cos\frac{\theta}{2},\ a\sin\frac{\theta}{2}\right), \qquad P = (0, p), \\ (w_1, v_1)(w_2, v_2) &= (w_1 w_2 - v_1^T v_2,\ w_1 v_2 + w_2 v_1 + v_1 \times v_2), \\ q^{-1} &= (w, -v) \quad (\|q\| = 1), \qquad P' = qPq^{-1}. \end{aligned} \tag{1.19}$$

a, p, v are 3×1; a is unit length. The vector part of pure quaternion P' is the rotated point. Unit quaternion entries are dimensionless; point coordinates retain their length unit.
Conditions: exact under the stated convention. For a nonunit quaternion the inverse is $(w, -v)/(w^2 + v^T v)$, not just its conjugate. PDF page 92 prints $q^{-1}Pq$: that gives the inverse/passive rotation relative to our active convention. Do not mix sandwich orders.

How to use it — rotate a point with a quaternion

1. Build or normalize the scalar-first quaternion.
2. Embed the point as $(0, p)$.
3. Apply Hamilton multiplication twice in the stated order.
4. Extract the vector part; check zero scalar part and preserved vector length.

See *Example 1.9*.

Worked example 1.9 — a quarter-turn in three representations

Source: original numerical companion to PDF pages 88–92.
For 90° about $a = (0, 0, 1)^T$, use $R_z(90^\circ)$ and $q = (\sqrt{2}/2, 0, 0, \sqrt{2}/2)$.

1. The matrix trace is one: $\theta = \arccos(0) = 90^\circ$.
2. The skew-entry formula returns $a = (0, 0, 1)$.
3. With $P = (0, 1, 0, 0)$, the first Hamilton product is $qP = (0, \sqrt{2}/2, \sqrt{2}/2, 0)$.
4. Multiplying by q^{-1} gives $P' = (0, 0, 1, 0)$, so the rotated point is $(0, 1, 0)^T$.
5. Rodrigues' formula and the elementary matrix agree. The opposite sandwich $q^{-1}Pq$ instead gives $(0, -1, 0)^T$.

Check 1.5 — is negative identity a 3D rotation?

The matrix $-I_3$ preserves lengths. Is it a proper rotation?

Solution

No: its determinant is -1. It is orthogonal but reverses orientation. Proper rotations require both orthogonality and determinant $+1$.

1.6 Reference frames and two worked quiz problems

A camera pose answers a question only after its coordinate frames are named. "The translation is $(2, 0, 0)$" is incomplete: from which frame to which other frame? This section makes the direction explicit, then uses it to solve the lecture's most relevant quiz problems.

Lecture source: `wk1_2_cs532s26_IntroToGeometry.pdf`, PDF pages 101–103, printed slides 102–104; supporting plane/rotation material on PDF pages 82–89.

1.6.1 Rigid motion is rotation plus translation

A ***rigid transformation*** preserves distances and angles. With column vectors, rotate a position into the target axes, then add the source origin's position expressed in the target frame.

Formula — rigid coordinate transform and inverse

$$\begin{aligned} p_B &= R_{B\leftarrow A}p_A + t_{B\leftarrow A}, \\ T_{B\leftarrow A} &= \begin{pmatrix} R & t \\ 0^T & 1 \end{pmatrix}, \qquad T_{A\leftarrow B} = \begin{pmatrix} R^T & -R^T t \\ 0^T & 1 \end{pmatrix}. \end{aligned} \tag{1.20}$$

p_A, p_B, t are 3×1 and use one length unit; R is a proper 3×3 rotation; T is 4×4. The inverse follows by subtracting t from p_B and multiplying by R^T.
Conditions: exact rigid transform. The inverse translation is $-R^T t$, not generally $-t$. Directions and normals under a rigid transform rotate without translation; positions translate as well.

How to use it — move a coordinate description

1. Label the source and target frames on the matrix.
2. Rotate the source-frame position and add the translation in target coordinates.
3. To reverse, apply the inverse with rotated negative translation.
4. Check that a source-frame origin maps to t and that forward/inverse composition is identity.

See *Example 1.10*.

Formula — compose compatible frame transforms

$$T_{C\leftarrow A} = T_{C\leftarrow B}T_{B\leftarrow A}, \qquad R_{CA} = R_{CB}R_{BA}, \quad t_{CA} = R_{CB}t_{BA} + t_{CB}. \tag{1.21}$$

All T matrices are 4×4; all R blocks are 3×3 and translations 3×1. Intermediate frame B matches between adjacent factors. The subscripts without arrows abbreviate the same target/source ordering.
Conditions: frames, time, object convention, and length units must agree. A product of numerically compatible matrices can still be geometrically meaningless if the frame path does not connect.

How to use it — follow the frame path

1. Write the desired coordinate journey, for example $A \to B \to C$.
2. Place the first operation on the right.
3. Invert any supplied transform whose direction opposes the journey.
4. Multiply and test the result on a known point.

See *Example 1.10*.

1.6.2 Quiz: recover camera pose through a shared object

Assigned source: Fall 2025 Quiz 1, questions 3–4, `cs532f25_quiz_1_solution.pdf`, PDF page 2. Its printed title is "RANSAC and Two-view Geometry." The probability questions 1–2 belong beside RANSAC in Lecture 3; the geometry questions belong here according to `quizByLec.md`.

Each static camera estimates the object's pose in its own coordinates. Assume both estimates describe the same rigid object frame at the same instant and in the same physical units.

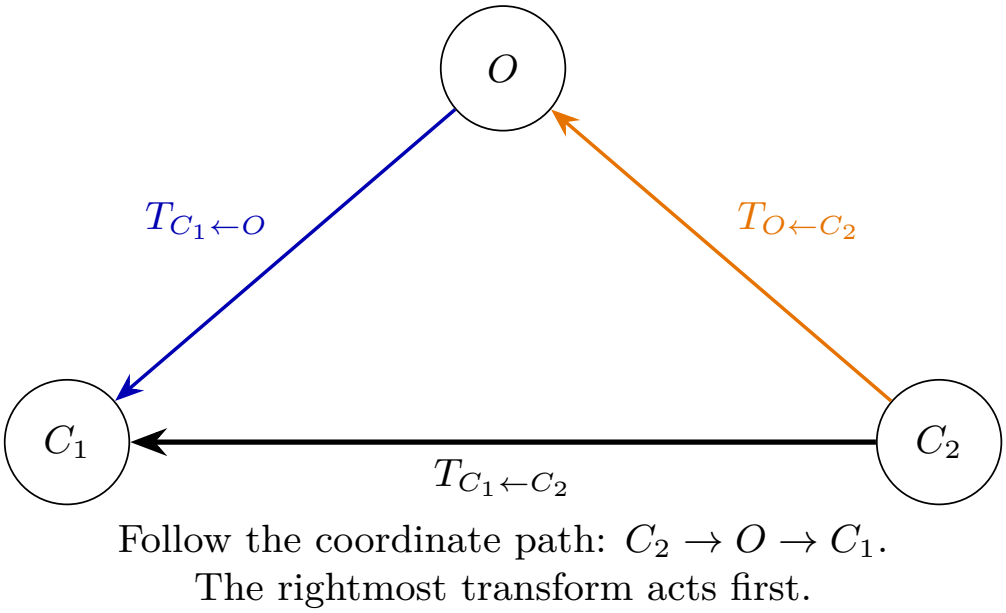


Fig. 1.11: Follow the transform directions through the shared object. This is a frame graph, not a drawing of physical camera positions. Original diagram for Fall 2025 Quiz 1, question 3.

Worked example 1.10 — Fall 2025 Quiz 1, question 3

Question: can a simultaneous pair of object-pose estimates determine the relative camera pose?
Answer: yes, under the shared-frame, time, and scale assumptions above.

1. The supplied maps are $T_{C_1 \leftarrow O}$ and $T_{C_2 \leftarrow O}$.
2. To express a camera-2 point in camera 1, first go from C_2 back to O by inverting the second map.
3. Then go from O to C_1 with the first map.

$$T_{C_1 \leftarrow C_2} = T_{C_1 \leftarrow O}(T_{C_2 \leftarrow O})^{-1}. \tag{1.22}$$

Writing each supplied pose as (R_i, t_i) gives $R_{12} = R_1 R_2^T$ and $t_{12} = t_1 - R_1 R_2^T t_2$. These are the composition and inverse rules above, not new assumptions.
Numerical illustration added here: take $R_1 = R_2 = I$, $t_1 = (1, 1, 4)^T$, and $t_2 = (-1, 1, 4)^T$. Then $R_{12} = I$ and $t_{12} = (2, 0, 0)^T$. The object origin in camera 2 is t_2; transforming it gives $t_2 + t_{12} = t_1$, which checks the direction.
If each pose estimate has an independent unknown scale, or uses a different ambiguous object orientation, this simple metric composition is not justified.

The boxed general composition formula is the canonical rule; the displayed quiz equation is its concrete specialization. Its 4×4 inputs and output have the same frame/unit conditions. Use the graph as the algorithm: invert the camera-2 object pose, multiply on the left by the camera-1 object pose, and verify on the object origin.

Worked example 1.11 — Fall 2025 Quiz 1, question 4

Question: can we estimate the plane through the object and both cameras?

Interpret "the object" as the origin of its estimated object frame, and choose camera 1 as the working coordinate frame.

1. Its center is $C_1 = (0, 0, 0)^T$.
2. Camera 2's center is $C_2 = t_{12}$. This works because the origin of frame C_2 maps to the translation column of $T_{C_1 \leftarrow C_2}$.
3. The object origin is $O = t_1$.
4. Form $n = (O - C_1) \times (C_2 - C_1)$, then $d = -n^T C_1 = 0$.

For the numerical illustration above, $n = (1, 1, 4) \times (2, 0, 0) = (0, 8, -2) \sim (0, 4, -1)$. The plane is $4y - z = 0$. It contains both camera centers and the object origin, since each gives zero on substitution.

Qualification to the supplied solution: this gives a unique plane only if the three positions are non-collinear. If the object origin lies on the camera baseline, the cross product is zero and there is no unique plane. An extended object is not necessarily wholly contained in this plane.

1.6.3 Quiz: tetrahedron face normals and a rotated object

Assigned source: Fall 2017 Quiz 9, problem 2(a–b), `cs532_f17_quiz9.pdf`, PDF page 2. This is supplementary practice for Lecture 1; it is not a high-probability exam prediction. Its depth-map problem 1 is previewed in the camera section and retained for deeper treatment in Lectures 3–4.

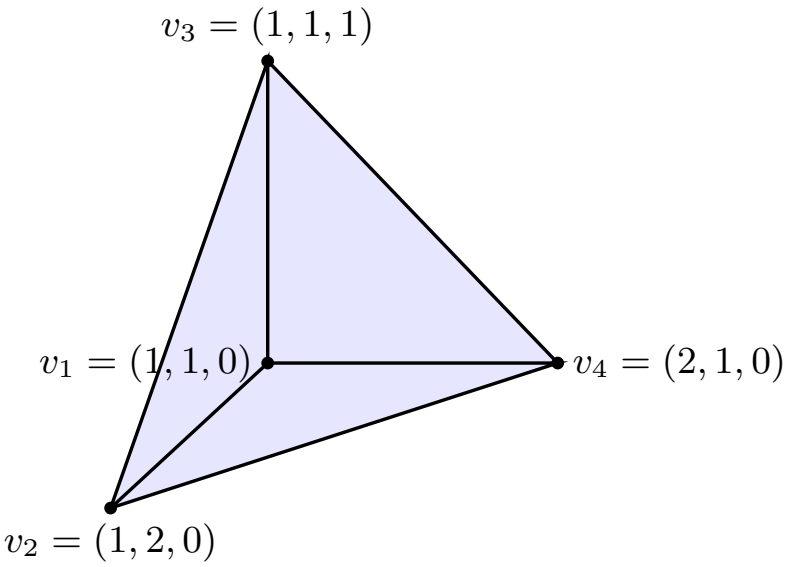


Fig. 1.12: The quiz tetrahedron. Coordinates are shown relative to the corner v_1 for drawing convenience; vertex labels give the actual quiz coordinates. Original schematic, not a metric perspective rendering.

Worked example 1.12 — Fall 2017 Quiz 9, problem 2(a)

The vertices are $v_1 = (1, 1, 0)$, $v_2 = (1, 2, 0)$, $v_3 = (1, 1, 1)$, and $v_4 = (2, 1, 0)$.

We choose **outward unit normals**. The problem does not specify orientation or normalization; declare them rather than silently mixing signed answers.

1. For face 123, $(v_2 - v_1) \times (v_3 - v_1) = (0, 1, 0) \times (0, 0, 1) = (1, 0, 0)$. This points toward the opposite vertex v_4, so flip it: $n_{123} = (-1, 0, 0)$.
2. Face 124 lies on $z = 0$, with the interior at positive z. Its outward normal is $(0, 0, -1)$.
3. Face 134 lies on $y = 1$, with the interior at larger y. Its outward normal is $(0, -1, 0)$.
4. For face 234, use $v_3 - v_2 = (0, -1, 1)$ and $v_4 - v_2 = (1, -1, 0)$. Their cross product is $(1, 1, 1)$. The opposite vertex lies on its negative side, so it already points outward. Normalize to $(1, 1, 1)/\sqrt{3}$.

A general orientation check is $n^T(v_{\text{opposite}} - v_{\text{face}}) < 0$ for an outward normal. Each normal must also have zero dot product with its face's two edge directions.

Worked example 1.13 — Fall 2017 Quiz 9, problem 2(b)

Rotate the object actively by $+90^\circ$ about the global z axis. Because this is a rigid rotation, normals rotate with R_z and do not translate.

The coordinate rule is $(x, y, z) \mapsto (-y, x, z)$. Applying it to every vertex gives $(-1, 1, 0)$, $(-2, 1, 0)$, $(-1, 1, 1)$, and $(-1, 2, 0)$.

Face	Original outward unit normal	Rotated outward unit normal
123	$(-1, 0, 0)$	$(0, -1, 0)$
124	$(0, 0, -1)$	$(0, 0, -1)$
134	$(0, -1, 0)$	$(1, 0, 0)$
234	$(1, 1, 1)/\sqrt{3}$	$(-1, 1, 1)/\sqrt{3}$

Check: recomputing each cross product from the rotated vertices gives the same oriented normal after normalization. Unit lengths and outward orientation are preserved. For a general affine deformation, use an inverse transpose for normals instead; multiplying by the forward matrix is special to rigid rotations (and needs normalization under uniform scale).

1.6.4 Where rigid transformations sit in the 3D hierarchy

The last geometry slide, PDF page 103, extends the 2D hierarchy to 3D. The homogeneous forms in (1.6) extend by making R, A 3×3 and t, v 3×1.

Family	3D continuous DOF	Important preserved properties
Proper rigid/Euclidean	6	Distances, angles, volume
Similarity	7	Angles and ratios of lengths
Affine	12	Parallelism, collinear ratios, volume ratios
Projective	15	Incidence and tangency where defined

A rigid motion has three rotation and three translation parameters. Adding uniform scale gives seven. An affine map has nine linear and three translation parameters. A projectivity has sixteen entries minus one common scale. Its matrix must be invertible. Projective notions such as the absolute conic, named in the hierarchy figures, motivate metric invariants but are developed with reconstruction upgrades in Lecture 6.

Check 1.6 — why not add translation to a normal?

A plane is moved three meters sideways without rotation. Does its normal acquire that translation?

Solution

No. A normal describes orientation, not position. The plane's offset changes, while its direction remains fixed. The homogeneous plane coefficients change through the inverse transpose, which accounts for the offset correctly.

1.7 Image formation: where does a 3D point appear?

You move a camera toward a stationary object. Does the object's image move toward the principal point, or away from it? This is the lecture's final hands-on question, and it ties projection to reference frames.

Lecture source: `wk1_2_cs532s26_IntroToGeometry.pdf`, PDF pages 104–114, printed slides 105–115. The introductory image-formation slides credit John Oliensis.

1.7.1 Start with a pinhole and similar triangles

An ideal **pinhole camera** sends each visible scene point along a ray through the camera center to an image plane. The physical sensor behind the pinhole receives an inverted image. We instead use a **virtual image plane** in front of it, which gives a positive focal-length convention and avoids carrying that inversion into every equation.

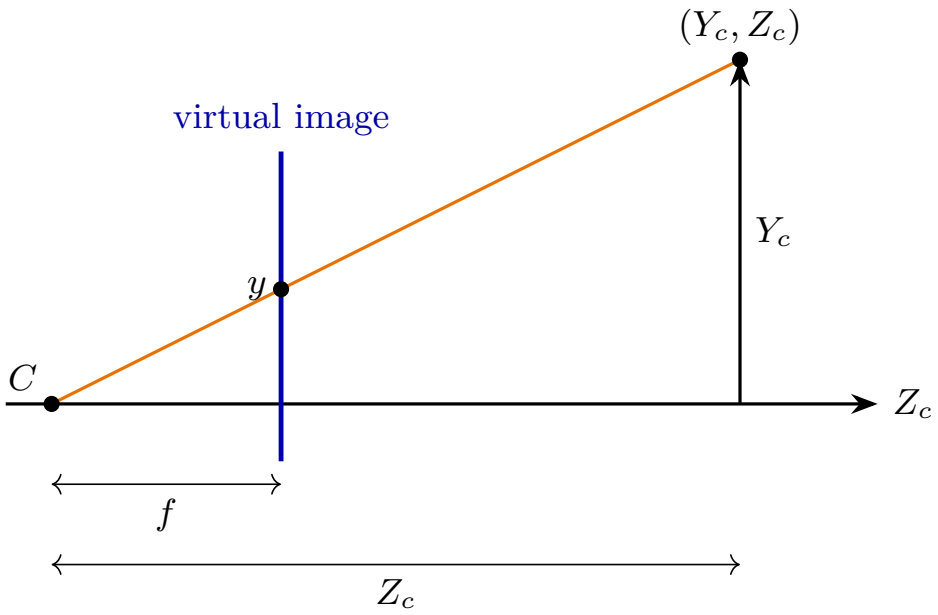


Fig. 1.13: A side-view similar-triangle construction. Original diagram developing PDF pages 105–107; those slide sections credit John Oliensis. The drawing's page orientation is schematic; the coordinate signs follow the labeled camera/image axes.

The small image triangle and large scene triangle share the ray angle. Their corresponding side lengths are proportional. The vertical ratio is $y/Y_c = f/Z_c$; the horizontal argument gives the same ratio for x.

Formula — perspective projection in camera coordinates

$$x = f\frac{X_c}{Z_c}, \qquad y = f\frac{Y_c}{Z_c}, \qquad \begin{pmatrix} x \\ y \\ f \end{pmatrix} = \frac{f}{Z_c}\begin{pmatrix} X_c \\ Y_c \\ Z_c \end{pmatrix}. \tag{1.23}$$

(X_c, Y_c, Z_c) is the point in camera coordinates. x, y, f are coordinates and focal length on the virtual image plane in the same length unit; the ratios are dimensionless. Normalized image coordinates are $(X_c/Z_c, Y_c/Z_c)$.
Conditions: exact ideal central projection, $f > 0$, and $Z_c \neq 0$. Visible points for this forward-facing model need $Z_c > 0$. Pixel coordinates require calibration; do not confuse physical focal length with focal length measured in pixels.

How to use it — project a point

1. Express the point in camera coordinates.

2. Check it is in front of the camera.
3. Divide the transverse coordinates by depth and multiply by focal length.
4. Check that points on one ray have the same image and that doubling depth at fixed transverse position halves the image displacement.

See *Example 1.14*.

Worked example 1.14 — perspective loses depth

Source: original numerical development of PDF pages 107–110.

Use a virtual-plane focal length of 2 length units and a point $(1, 0.5, 10)$ in camera coordinates.

1. $x = 2(1/10) = 0.2$ and $y = 2(0.5/10) = 0.1$ in image-plane length units.
2. The point $(2, 1, 20)$ gives the same $(0.2, 0.1)$ because every scene coordinate was scaled equally.
3. The point $(1, 0.5, 20)$ instead gives $(0.1, 0.05)$: it moved deeper without increasing its transverse displacement.

One image coordinate does not uniquely determine a 3D point. The inverse is a ray unless depth or another geometric constraint is supplied.

1.7.2 Projection properties and their exceptions

The lecture says that lines project to lines and polyhedra project to polygons (PDF 108–109). The underlying reason is that a 3D line together with the camera center defines a plane; that plane intersects the image plane in an image line.

There is a degenerate case: a 3D line through the camera center projects to a single image point, wherever its visible projection is defined. A point at the camera center has no valid projection at all. Occlusion also determines which projected edges are visible. These qualifications prevent an ideal geometric statement from being mistaken for a guarantee about every photograph.

The "distant objects are smaller" example on PDF page 110 is about equal physical objects at different depths: their projected sizes vary inversely with depth under the same camera. Changing focal length at the same time changes that comparison.

1.7.3 Separate camera intrinsics from camera pose

Intrinsics turn normalized camera coordinates into pixel coordinates. **Extrinsics** turn world coordinates into camera coordinates. They answer different questions and have different units.

Formula — calibrated pixel projection

$$K = \begin{pmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}, \qquad P = K \begin{pmatrix} R & t \end{pmatrix}, \tag{1.24}$$
$$\lambda \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = P X_w, \qquad X_c = R X_{w,\mathrm{E}} + t, \qquad C = -R^T t.$$

K is 3×3; f_x, f_y and skew s are in pixels, and (c_x, c_y) is the ***principal point*** in pixels. X_w is a homogeneous 4×1 world point; $X_{w,\mathrm{E}}$ is its Euclidean 3×1 form. R, t map world to camera, so P is 3×4. C is the camera center in world coordinates.
Conditions: ideal undistorted camera, finite valid point, and a declared world-to-camera convention. With a world point normalized to last coordinate one, this calibration gives $\lambda = Z_c$. P is defined up to nonzero scale. A calibrated finite camera has rank three; its nullspace is the homogeneous camera center.

How to use it — world point to pixel

1. Compute $X_c = R X_{w,\mathrm{E}} + t$ and check $Z_c > 0$.
2. Multiply by K, or multiply $P X_w$ directly.
3. Divide the first two output entries by the third.
4. Check image bounds separately: a point in front can still project outside the sensor.
5. Verify the center relation by $RC + t = 0$.

See *Example 1.15*.

Expanding the first two rows makes the meaning explicit:

$$u = f_x X_c / Z_c + s Y_c / Z_c + c_x, \qquad v = f_y Y_c / Z_c + c_y.$$

The offsets place the optical-axis intersection at (c_x, c_y), not necessarily at image coordinate zero. Pixel focal length is a physical focal length divided by pixel pitch; different pixel scales can give different f_x, f_y. Detailed calibration and homography estimation follow in Lecture 3.

Convention translation for PDF page 113: the slide writes $K[R^T \mid -R^T t]$ when R, t describe the camera's pose in world coordinates. In our notation call those $R_{w\leftarrow c}, C$. Then the world-to-camera map is $[R_{w\leftarrow c}^T \mid -R_{w\leftarrow c}^T C]$. It is the same geometry, expressed with a different meaning for the symbol R. The slide's "2-D to 3-D" description should not suggest that P uniquely inverts projection: P maps 3D homogeneous points to 2D homogeneous image points.

1.7.4 Work the actual camera-motion exercise

PDF page 114 asks us to choose visible points for a camera with focal length 500 and a 500×500 image, move the camera forward by two units, and then consider rotating/translating points. The slide leaves the points, principal point, and precise sequencing open. We must declare them.

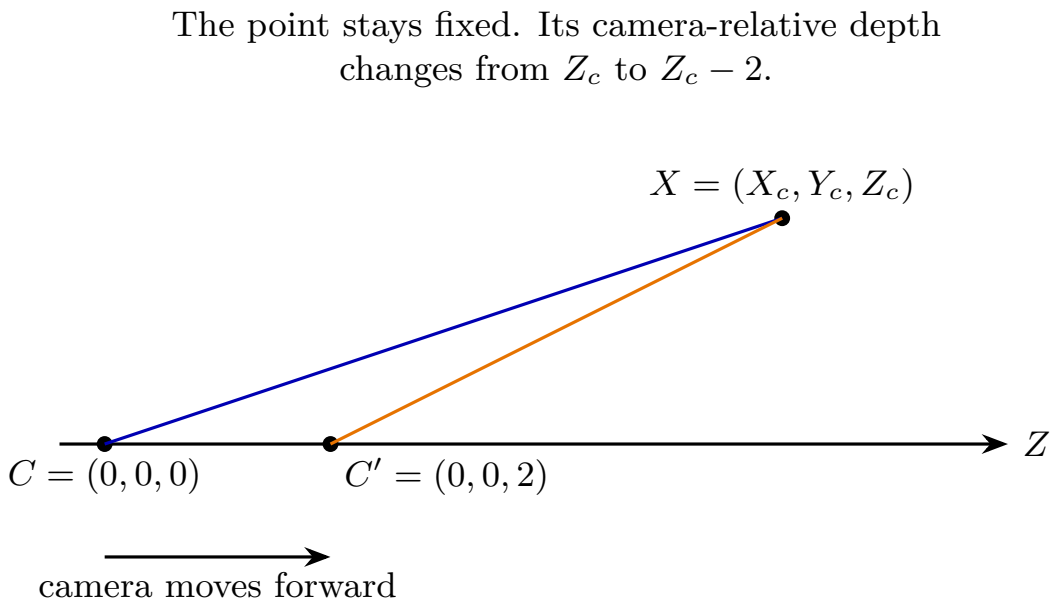


Fig. 1.14: Move the camera, not the object: the world point stays fixed, but its camera-relative depth decreases. Original solution diagram for PDF page 114, printed slide 115.

Worked example 1.15 — the lecture's camera-motion problem

Source: numerical solution adapted from PDF page 114. All values added to complete the setup are stated here.

Assume square pixels, zero skew, $f_x = f_y = 500$ pixels, and principal point $(250, 250)$. Initially the camera is at the world origin with $R = I$. Choose $X_w = (1, 0.5, 10)$ in an arbitrary consistent length unit. Its image is inside the 500×500 sensor.

1. Initially, $(u, v) = (250 + 500/10, 250 + 250/10) = (300, 275)$ pixels.
2. Move the camera center to $C' = (0, 0, 2)$ without rotating it. The stationary point now has camera coordinates $(1, 0.5, 8)$.
3. The new pixel is $(312.5, 281.25)$. Its displacement from the principal point increases because the point is closer.
4. For a separate branch starting again from the original setup, rotate the **point** by 45° about the camera's z axis and then translate it five units farther along z.
5. The new point is $(0.5/\sqrt{2}, 1.5/\sqrt{2}, 15)$. Projection gives $(261.785, 285.355)$ pixels after rounding.

If the operations were meant cumulatively after the camera move, the rotated/transformed point would have depth $8 + 5 = 13$ in that moved camera, giving approximately $(263.598, 290.795)$ instead. A complete answer must specify the sequence.

For a general forward camera displacement Δ, the normalized transverse displacement changes from X_c/Z_c to $X_c/(Z_c - \Delta)$; visible points need $Z_c - \Delta > 0$. A point on the optical axis does not move away from the principal point. Translating the object farther away instead gives denominator $Z_c + \Delta$ and moves its image inward. Camera motion and scene motion require inverse coordinate relationships, as PDF page 111 emphasizes.

1.7.5 A depth-map bridge to the assigned quiz

Source: Fall 2017 Quiz 9, problem 1, `cs532_f17_quiz9.pdf`, PDF page 1. The quiz asks how two depth maps can become one common-frame point cloud. This is a supplementary preview; depth estimation and fusion algorithms are beyond Lecture 1.

Formula — axial-depth backprojection and frame conversion

$$q = K^{-1}\begin{pmatrix} u \\ v \\ 1 \end{pmatrix}, \qquad X_c = \frac{Z_c}{q_3} q, \qquad X_w = R^T(X_c - t). \tag{1.25}$$

K is 3×3, q, X_c, X_w, t are 3×1, and Z_c is ***axial depth*** in the camera's forward direction. For the standard intrinsic matrix $q_3 = 1$. The final equation assumes R, t map world to camera.
Conditions: known calibration and camera poses in a common metric frame, invertible K, valid positive depth, and distortion handled. If depth means Euclidean range r along the ray, use $X_c = rq/\|q\|$ instead. Mixing range and axial depth produces incorrect geometry.

How to use it — turn depth maps into common-frame points

1. For each valid depth pixel, use that camera's calibration to recover a viewing direction.
2. Scale it according to the depth convention.
3. Transform the camera-coordinate point into the shared world frame.
4. Repeat for both cameras and combine the points. Combining does not automatically denoise or resolve occlusion.

See *Example 1.16*.

Worked example 1.16 — Fall 2017 Quiz 9, problem 1: preview solution

The required information is each camera's intrinsics, distortion model or undistorted pixels, a defined depth convention and scale, and extrinsics expressed relative to one common frame. The boxed procedure supplies the construction.
For a small numerical illustration added here, let both cameras use the K from Example 1.15. Camera 1 is at the origin and camera 2 at $(2, 0, 0)$, both with identity orientation.

1. Camera 1's pixel $(300, 250)$ with axial depth 10 backprojects to $(1, 0, 10)$ in camera 1 and world coordinates.
2. Camera 2's pixel $(200, 250)$ at depth 10 backprojects to $(-1, 0, 10)$ in camera 2.
3. Camera 2 has world-to-camera translation $t = (-2, 0, 0)$. Applying $X_w = X_c - t$ gives $(1, 0, 10)$, the same world point.

The two camera centers and their rays to this common point form the illustrative geometry: draw the centers at $(0, 0, 0)$ and $(2, 0, 0)$ and connect both to $(1, 0, 10)$. The shared-object frame graph and the projection-ray figures show the two ingredients separately. A dedicated ray sketch follows.

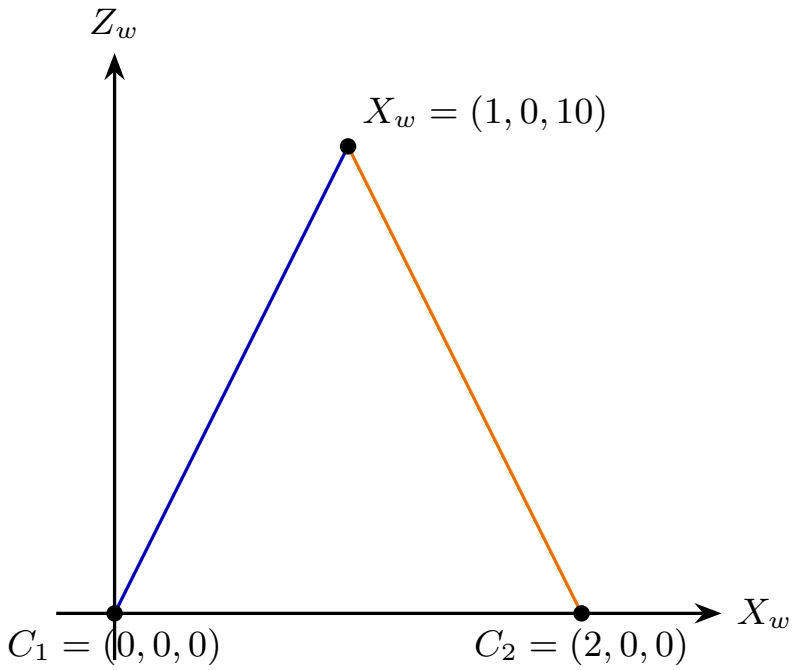


Fig. 1.15: Both backprojected points coincide after conversion to world coordinates. Original schematic for Fall 2017 Quiz 9, problem 1; the optical-axis depth and coordinates come from Example 1.16.

Check 1.7 — is an off-image point behind the camera?

A point has positive camera depth but projects to $u = 700$ in a 500-pixel-wide image. Which condition failed?

Solution

It is in front of the camera but outside the image bounds. Positive depth and sensor visibility are separate tests. A valid homogeneous projection alone does not establish that the camera actually measured the point.

1.8 Foundation quiz practice

Apply the geometry and camera formulas from this chapter to **Spring 2020 Quiz 1**. The supplied quiz map assigns it to Lecture 2; its solutions live here because they test Lecture 1 concepts.

Use (1.1) and (1.2) for Problems 1 and 3, and (1.24) for Problem 2.

1.8.1 Quiz Problem 1: are three points collinear?

Combining the two referenced rules gives the quiz's compact three-point test.

Formula — collinearity test

$$s = c^T(a \times b), \qquad s = 0 \quad \text{for collinear points.} \tag{1.26}$$

a, b, c are nonzero homogeneous 3×1 point vectors. The zero/nonzero result is unchanged by rescaling them. This is an exact determinant test. For measured points, a small unnormalized s is not a meaningful distance threshold.

Check collinearity without accepting an invalid line

1. Check that all point vectors are nonzero.
2. Form $a \times b$. If it vanishes, choose another distinct pair.
3. Dot that line with the third point; zero means incidence.

If all three represent the same point, no unique joining line exists. Any line through that point contains the repeated inputs. See Example 1.17.

Worked example 1.17 — the dot/cross quiz

Source: `cs532s20_quiz1.pdf`, Problem 1; numerical illustration is original.
Take $a = (0, 0, 1)^T$, $b = (2, 0, 1)^T$, and $c = (5, 0, 1)^T$. Then $a \times b = (0, 2, 0)^T$ and $c^T(a \times b) = 0$: all lie on $y = 0$.
The quiz asks when the test fails. If $b = 2a$, the cross product is zero and cannot define a line. Its dot product with **any** c is zero. This fails to construct a unique line. Mathematically the determinant still correctly reports linear dependence, and a line through the repeated point and c exists. Distinguish **collinearity as existence** from **construction of a unique line**.

1.8.2 Quiz Problem 2: what size is a camera matrix?

Worked example 1.18 — dimensions before arithmetic

Source: `cs532s20_quiz1.pdf`, Problem 2, both parts.

A homogeneous 3D point has four entries; an image point has three. Therefore M is 3×4, since $(3 \times 4)(4 \times 1) = 3 \times 1$. See the general mapping in (1.24).

For $X = (X_1, X_2, X_3, X_4)^T$, the ordinary coordinates are

$$u = \frac{m_{11}X_1 + m_{12}X_2 + m_{13}X_3 + m_{14}X_4}{m_{31}X_1 + m_{32}X_2 + m_{33}X_3 + m_{34}X_4},$$

$$v = \frac{m_{21}X_1 + m_{22}X_2 + m_{23}X_3 + m_{24}X_4}{m_{31}X_1 + m_{32}X_2 + m_{33}X_3 + m_{34}X_4}.$$

For a finite world point choose $X_4 = 1$. Both fractions require a nonzero denominator. If MX is nonzero with last entry zero, the image point is ideal; if $MX = 0$, projection is undefined, as at the camera center. Nonzero rescaling of X or M changes neither fraction.

1.8.3 Quiz Problem 3: how many intersections?

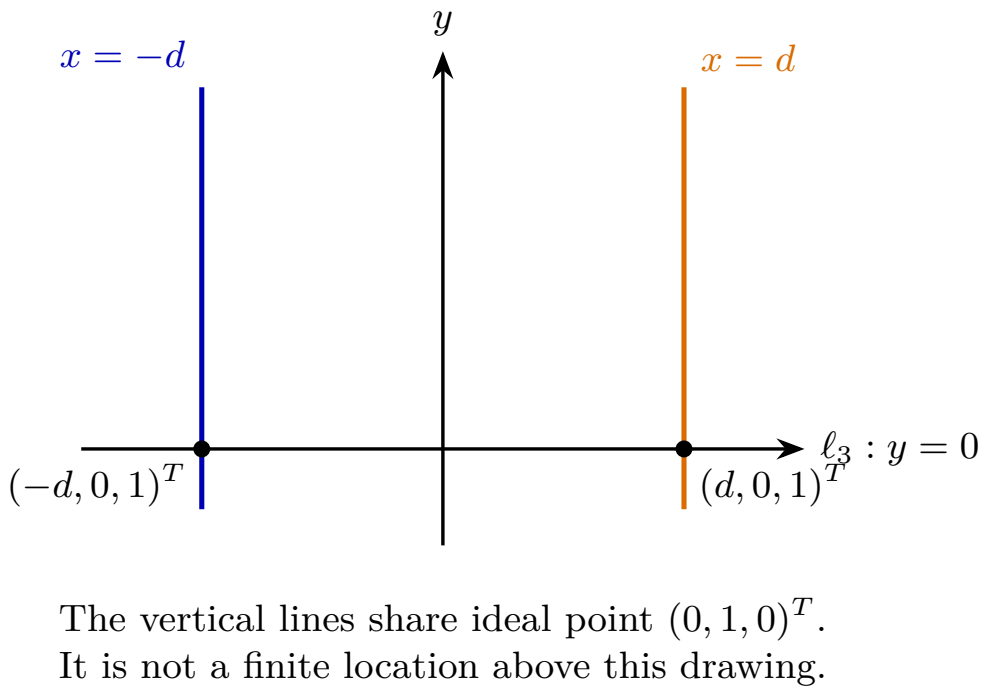


Fig. 1.16: The third intersection is at infinity. It has no ordinary position on this finite drawing. Original diagram of Quiz Problem 3.

Worked example 1.19 — count in the projective plane

Source: `cs532s20_quiz1.pdf`, Problem 3, both parts.
Assume $d > 0$ for two distinct vertical lines each distance d from the origin. Choose $\ell_1 = (1, 0, -d)^T$ for $x = d$, $\ell_2 = (1, 0, d)^T$ for $x = -d$, and $\ell_3 = (0, 1, 0)^T$ for $y = 0$.
Use (1.2) for each pair:

1. $\ell_1 \times \ell_3 = (d, 0, 1)^T$ gives $(d, 0)$.
2. $\ell_2 \times \ell_3 = (-d, 0, 1)^T$ gives $(-d, 0)$.
3. $\ell_1 \times \ell_2 = (0, -2d, 0)^T \sim (0, 1, 0)^T$ is the vertical ideal point.

There are **three distinct projective intersections**, of which **two are finite**. The ideal point happens to have the same entries as the horizontal line's coefficients; its geometric type comes from how it is used. Setting $d = 0$ violates the distinct-line assumption.

Check 1.8 — test a point off the line

Replace c in Example 1.17 by $(5, 1, 1)^T$. What changes?

Answer

The dot product is 2: the three points are not collinear.

1.9 Review: choose the right geometric operation

The most useful formula is the one whose inputs, coordinate conventions, and assumptions match the problem. Before calculating, say what geometric object you have and what you want to recover.

1.9.1 Formula and source checklist

The PDF-page ranges refer to `wk1_2_cs532s26_IntroToGeometry.pdf`. Equation links lead to the canonical formula boxes and application procedures.

Topic / operation	Formula or explanation	Source PDF pages
Motivation, reconstruction pipeline, ambiguity, application examples	*Why reconstruct a world from photographs?*	2–61
Homogeneous scale, finite/ideal points	*Points, lines, and the meaning of infinity* and (1)	65–71
Point–line incidence	(1.1)	65
Joins and intersections	(1.2)	67
Parallel lines and infinity	(1.3)	68–71
2D rotation and pivot transforms	(1.4), (1.5)	72–77
Transformation families and invariants	(1.6)	78, 103
Point, line, and plane transforms	(1.7)	79, 81–82
Planes, distance, three-point construction	(1.8), (1.9)	82–84
Point/plane nullspaces and cofactors	(1.10), (1.11)	83–85
Plücker representation and line distances	(1.12), (1.13)	86–87
Axis rotations and inverse	(1.14), (1.15)	88–90
Ordered angles and fixed/body axes	(1.16)	93–100
Axis–angle and computational supplement	(1.17), (1.18)	91
Quaternion operations and conventions	(1.19)	92
Rigid transforms and frame composition	(1.20), (1.21)	101–102
Virtual-plane projection	(1.23)	105–110
Extrinsics, intrinsics, projection matrix	(1.24)	111–113
Camera-motion hands-on task	*Example 1.15*	114
Depth-map backprojection: quiz extension	(1.25)	Quiz, not a slide formula

The epipolar constraint and disparity formula embedded in the workflow previews on PDF pages 25–26 are explicitly deferred to Lecture 4, where their variables and assumptions will be developed.

The administrative title, policies, logistics, and reading-list slides are recorded in the coverage ledger but do not become mathematical exercises. The rank/SVD and Rodrigues explanations fill explanatory steps needed to understand the lecture; they are identified as supplements where they go beyond its printed formulas.

1.9.2 Rework the assigned quizzes without the solutions

Source	Location	What to check
Fall 2025 Quiz 1, Q3	*Example 1.10*	Frame path, inversion, shared object frame/scale/time
Fall 2025 Quiz 1, Q4	*Example 1.11*	Camera centers in one frame; noncollinearity
Fall 2017 Quiz 9, P2(a)	*Example 1.12*	Cross products, orientation, unit normalization
Fall 2017 Quiz 9, P2(b)	*Example 1.13*	Declared active rotation; normals remain perpendicular
Fall 2017 Quiz 9, P1	*Example 1.16*	Supplement: calibration, axial depth/range, common frame

Fall 2025 Quiz 1, Q1–Q2, remains assigned to Lecture 3's RANSAC/probability explanation. The Spring 2020 homogeneous-geometry quiz remains in Lecture 2; use its questions later as a transfer test rather than counting them as newly introduced Lecture 1 quiz material.

1.9.3 Common mistakes to catch before you calculate

- Dividing a homogeneous coordinate by zero or treating the zero vector as a point.
- Calling an image vanishing point an ideal image point without checking its coordinates.
- Rotating about the origin when a different pivot was specified.
- Applying the point matrix directly to line/plane coefficients.
- Inferring a unique plane from collinear points.
- Treating every length-preserving matrix as a proper rotation.
- Using ordered-angle names without declaring axes and multiplication order.
- Mixing active/passive quaternion sandwiches.
- Negating a translation without rotating it when inverting a rigid transform.
- Treating camera pose and world-to-camera extrinsics as identical arrays.
- Assuming image-plane length, pixel coordinates, axial depth, and range use interchangeable units.

1.9.4 A short transfer problem

Exercise 1.1 — combine frames, planes, and projection

A world point is $X = (2, 1, 6)^T$. A camera has center $C = (1, 0, 2)^T$, identity orientation, $f_x = f_y = 400$ pixels, zero skew, and principal point $(200, 150)$.

1. Write the world-to-camera translation and camera coordinates of X.
2. Project X to pixels.
3. Find a plane through the camera center, the world origin, and X.
4. State when the plane-construction step would fail.

Solution

1. $t = -C = (-1, 0, -2)^T$, so $X_c = (1, 1, 4)^T$.
2. $u = 200 + 400(1/4) = 300$ and $v = 150 + 400(1/4) = 250$ pixels.
3. Use the world origin as anchor: $C \times X = (-2, -2, 1)^T$. Thus $-2x - 2y + z = 0$. Both C and X satisfy it; the origin does too.
4. It fails to determine a unique plane if the origin, camera center, and point are collinear, or if repeated points remove an independent direction.

You are ready to move on when you can explain the geometry before choosing a formula, solve these examples without copying their steps, and state each answer's assumptions and degeneracies. Lecture 2 will revisit the projective and rotation language and extend it toward image transformations and estimation.

Finish with the *TLDR and short quiz* for a quick self-check.

1.10 TLDR

Lecture 1 in one sentence: images lose depth, so reconstructing a scene requires geometry, coordinate frames, and careful use of multiple measurements.

1.10.1 The big picture

The reconstruction pipeline is **match features → estimate cameras and 3D points → estimate dense depth → build a representation**. A convincing picture is not proof of correct geometry: lighting, occlusion, repeated patterns, and noise can mislead us.

1.10.2 The rules worth remembering

1. **Homogeneous coordinates ignore overall scale.** $(2, 4, 2)^T$ and $(1, 2, 1)^T$ represent the same point. Divide by the last coordinate to get ordinary coordinates only when it is nonzero. A zero last coordinate means an ideal point; the all-zero vector is invalid. A scene direction can project to a finite image vanishing point. See *Points, lines, and the meaning of infinity*.

2. **Dot products test incidence; cross products construct geometry.** A point lies on a line when $\ell^T x = 0$. Join two distinct image points with $x \times y$; intersect two distinct image lines with $\ell_1 \times \ell_2$. Parallel lines meet at an ideal point. See (1.2).

3. **Choose the transformation you actually need.** Rigid motion preserves distances and angles; similarity adds uniform scale; affine maps preserve parallelism; projective maps preserve incidence but can change parallelism. Points transform with H; line and plane coefficients use its inverse transpose. To rotate about a pivot, **subtract the pivot → rotate → add it back**. See *Move points without losing the geometry*.

4. **Three noncollinear points determine a plane.** Use $n = (B - A) \times (D - A)$ and $d = -n^T A$ in $n^T p + d = 0$ for a point p. Collinear points give no unique plane. A unit normal gives orientation; its sign depends on your chosen inward/outward convention. Under a rigid rotation, normals rotate without translation. See *Points, planes, and lines in 3D*.

5. **A rotation is more than a square matrix.** A proper rotation satisfies $R^T R = I$ and $\det R = +1$, so $R^{-1} = R^T$. Rotation order matters. Matrices, ordered angles, axis–angle, and unit quaternions describe rotations, but their conventions must agree. Here active quaternion rotation is qPq^{-1}. See *Rotations: several descriptions of the same motion*.

6. **Label every coordinate frame.** For $p_B = Rp_A + t$, undo the motion with $p_A = R^T(p_B - t)$. In a matrix product the rightmost transform acts first. To go from camera 2 through a shared object to camera 1, use $T_{C_1 \leftarrow O}(T_{C_2 \leftarrow O})^{-1}$. See *Reference frames and two worked quiz problems*.

7. **Projection divides by depth.** In camera coordinates, normalized image coordinates are $(X_c/Z_c, Y_c/Z_c)$. Intrinsics convert these to pixels; extrinsics convert world coordinates to camera coordinates. Together, $x \sim K[R \mid t]X_w$. Require positive depth for a point in front, then check image bounds separately. See *Image formation: where does a 3D point appear?*.

8. **Camera motion changes the point's camera coordinates.** Moving the camera forward by Δ changes a stationary point's depth to $Z_c - \Delta$. Off-axis points move farther from the principal point while they remain in front. Moving the object farther away does the opposite. Backprojection needs calibration and a specified depth convention: axial depth and distance along the ray are different. See *Example 1.15*.

Also recognize: a nullspace solves homogeneous incidence equations; a finite Plücker line stores direction d and moment $p \times d$. The *formula checklist* links to these less frequently used constructions and their conditions.

Before calculating: name the frames, check units, choose the formula, and test whether the answer makes geometric sense.

1.10.3 Short quiz

Try these without looking at the answers. These are original practice questions based on Lecture 1; the assigned course quizzes are worked through in *Reference frames and two worked quiz problems*.

1. **Depth:** why can one image pixel correspond to many 3D points?
2. **Coordinates:** what ordinary point does $(6, 9, 3)^T$ represent? Can you similarly normalize $(1, 0, 0)^T$?
3. **Intersection:** find the homogeneous intersection of $x = 2$ and $y = 3$, using line vectors $(1, 0, -2)^T$ and $(0, 1, -3)^T$.
4. **Pivot:** rotate $(3, 1)$ by $+90^\circ$ about $(2, 1)$ in Cartesian axes with y upward. Where does it land?
5. **Plane:** find a normal and plane equation through $(0, 0, 0)$, $(1, 0, 0)$, and $(0, 1, 0)$. When would the three-point method fail?
6. **Rotation:** a matrix satisfies $R^T R = I$ but has determinant -1. Is it a proper rotation?
7. **Frames:** two object-to-camera poses have identity rotations and translations $t_1 = (1, 1, 4)^T$, $t_2 = (-1, 1, 4)^T$. What is the translation from camera 2 to camera 1?
8. **Camera:** use zero skew, $f_x = f_y = 100$ pixels, and principal point $(0, 0)$. Project camera-coordinate point $(2, 1, 5)^T$. Then move the camera one unit forward without rotating it while the point stays fixed. What is the new pixel?

1.10.4 Answers

Show answers and short explanations

1. **A pixel identifies a ray, not a depth.** Different positive depths along that ray produce the same image point.
2. $(2, 3)$**.** Divide the first two entries by three. $(1, 0, 0)^T$ is an ideal point; dividing by its last coordinate is invalid.
3. $(2, 3, 1)^T$**.** The cross product gives this vector. Substitution into both line equations gives zero.
4. $(2, 2)$**.** The offset from the pivot is $(1, 0)$. Rotate it to $(0, 1)$, then add the pivot back.
5. $n = (0, 0, 1)^T$**, so** $z = 0$**.** Cross the two edge vectors. Its negative is equally valid for the unoriented plane. Collinear or repeated inputs cannot determine a unique plane.
6. **No.** It is orthogonal but includes a reflection. A proper rotation needs determinant $+1$.
7. $(2, 0, 0)^T$**.** With both rotations equal to identity, the relative translation is $t_1 - t_2$. Check: camera 2's object coordinates plus this translation equal camera 1's.

8. **Initially** $(40, 20)$ **pixels; afterward** $(50, 25)$**.** First compute $(100 \cdot 2/5, 100 \cdot 1/5)$. Moving the camera forward reduces depth from five to four, so the image moves away from the principal point.

CHAPTER

TWO

LECTURE 2: PROJECTIVE GEOMETRY AND IMAGE WARPING

How can a tilted photograph become a view from above? The basketball-court task turns geometry into a practical tool: find a mapping, then use it to create a new image.

This chapter teaches the material added beyond Lecture 1: quaternion conversions and interpolation, planar rectification, image sampling, robust estimation, lens correction, and two-view constraints.

Prerequisites: *Lecture 1* owns homogeneous geometry, transformation families, rotations, frames, and camera projection. The source slides repeat those topics; the textbook links to their original explanations. Spring 2020 Quiz 1 remains attributed to Lecture 2 in the *quiz index*, with its foundation solutions collected in *Lecture 1*.

2.1 Reading route

1. *Quaternion conversions*: convert rotation descriptions and interpolate orientations.
2. *Homographies*: straighten a photographed plane and calculate cross-ratios.
3. *Image warping*: sample the source image to fill a destination grid.
4. *Estimate a reliable mapping*: DLT, normalization, and RANSAC.
5. *Distortion and two-view geometry*: handle lens bending and constrain matches across views.
6. *Review and formula checklist*, then the final *TLDR and short quiz*.

Source: `cs532_f25_Week2.pdf`, 129 PDF pages. Printed labels equal PDF pages through page 43; pages 44–49 have no visible label; from PDF page 50 onward, printed labels are one greater. Notes below use **PDF pages** unless stated otherwise. The authoring coverage register records which repeated slides are already covered in Lecture 1.

2.2 Quaternions: convert without changing the rotation

Prerequisite: (1.19) defines the quaternion convention. This section adds conversion and interpolation.

Source: PDF pages 42–49. Page 48 uses the transpose/passive matrix relative to our convention. Page 49 omits a necessary $+1$ inside a square root. The formulas below use one consistent convention; see the *Errata and assumptions to verify*.

2.2.1 From a quaternion to a matrix

Write the scalar-first unit quaternion as $q = (w, x, y, z)$. Collecting the point-coordinate coefficients in Lecture 1's quaternion rotation gives the matrix below. Here x, y, z name quaternion components.

Formula — quaternion to active rotation matrix

$$R(q) = \begin{pmatrix} 1 - 2(y^2 + z^2) & 2(xy - wz) & 2(xz + wy) \\ 2(xy + wz) & 1 - 2(x^2 + z^2) & 2(yz - wx) \\ 2(xz - wy) & 2(yz + wx) & 1 - 2(x^2 + y^2) \end{pmatrix}. \tag{2.1}$$

q has four unitless entries; R is 3×3. This form assumes **unit length**.

Turn an orientation into a matrix

1. Reject a zero quaternion; otherwise normalize it.
2. Substitute its scalar-first components.
3. Check $R^T R = I$, $\det R = 1$, and a familiar vector.

Use Example 2.1 to check the signs.

2.2.2 From a matrix back to a quaternion

Taking the trace gives $\operatorname{tr} R = 4w^2 - 1$. That explains the square root and the missing $+1$ in the slide.

Formula — matrix to quaternion, trace branch

$$\begin{aligned} w &= \tfrac{1}{2}\sqrt{1 + \operatorname{tr} R}, \\ (x, y, z) &= \frac{(R_{32} - R_{23},\ R_{13} - R_{31},\ R_{21} - R_{12})}{4w}. \end{aligned} \tag{2.2}$$

Indices are one-based. R must be a proper active rotation. This branch chooses $w \geq 0$ and requires w safely away from zero. Near a half-turn, division by w is unstable.
Use the largest vector component instead. For example, when x is largest,

$$x = \tfrac{1}{2}\sqrt{1 + R_{11} - R_{22} - R_{33}}, \quad (w, y, z) = \frac{(R_{32} - R_{23},\ R_{12} + R_{21},\ R_{13} + R_{31})}{4x}.$$

The y and z branches follow by cyclic permutation. Normalize the result; the overall sign is arbitrary.

Recover an orientation safely

1. Validate the matrix as a proper rotation.
2. Use the trace branch away from a half-turn; near a half-turn, use the largest vector-component branch.
3. Normalize and rebuild $R(q)$; compare matrices rather than quaternion signs.

See Example 2.1. Clamp small rounding errors only after validating the input.

Worked example 2.1 — cycle the axes, then test a half-turn

Source: original numerical companions to PDF pages 48–49.

For 120° about $a = (1,1,1)^T/\sqrt{3}$, the quaternion is $q = (1/2, 1/2, 1/2, 1/2)$. Substitution gives $R = \begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}$. It cycles the coordinate axes: $e_x \to e_y \to e_z \to e_x$.

The trace is zero. Recover $w = 1/2$ and $x = y = z = 1/(4w) = 1/2$. Omitting $+1$ would give $w = 0$ and make the remaining divisions fail. This tests a different rotation from Lecture 1's quarter-turn.

For $R = \mathrm{diag}(1, -1, -1)$, the trace branch gives $w = 0$, so do not divide by it. The largest-component branch gives $x = 1$, $w = y = z = 0$: a 180° turn about x.

2.2.3 Interpolation

The slides mention smooth interpolation. **SLERP** (spherical linear interpolation) follows the unit-quaternion sphere instead of independently changing Euler angles:

Formula — interpolate two orientations

$$q(t) = \frac{\sin((1-t)\Omega)}{\sin\Omega} q_a + \frac{\sin(t\Omega)}{\sin\Omega} q_b, \qquad \cos\Omega = q_a^T q_b, \quad 0 \le t \le 1. \tag{2.3}$$

This is an explanatory companion to PDF page 45, not a printed formula. Both endpoints are unit 4-vectors. If their dot product is negative, negate one endpoint first to select a shortest rotation path. Constant change in t gives constant angular speed. For nearly equal endpoints, use normalized linear interpolation to avoid a small $\sin\Omega$ denominator.

Find an in-between orientation

1. Normalize endpoints, align signs, and clamp their computed dot product to $[-1, 1]$.
2. Calculate Ω and the weights, or use the near-equal fallback.
3. Normalize and check the endpoints at $t = 0, 1$.

Between identity $(1,0,0,0)$ and a 90° z rotation, $t = 1/2$ gives $(\cos 22.5^\circ, 0, 0, \sin 22.5^\circ)$: a 45° rotation.

Check 2.1 — do opposite quaternions cancel?

What happens if you average q and $-q$ directly?

Answer

You get the zero vector, which cannot represent an orientation. Align signs first; both endpoints describe the same rotation.

2.3 Homographies: straighten a photographed plane

A basketball court is rectangular, but its photograph is not. A ***homography*** maps one projective plane to another, letting us turn the photographed floor into a rectangle. It also preserves straight lines, so the court markings stay straight.

Source: PDF pages 62–71, printed slides 63–72. The example below develops the actual court-warping assignment.

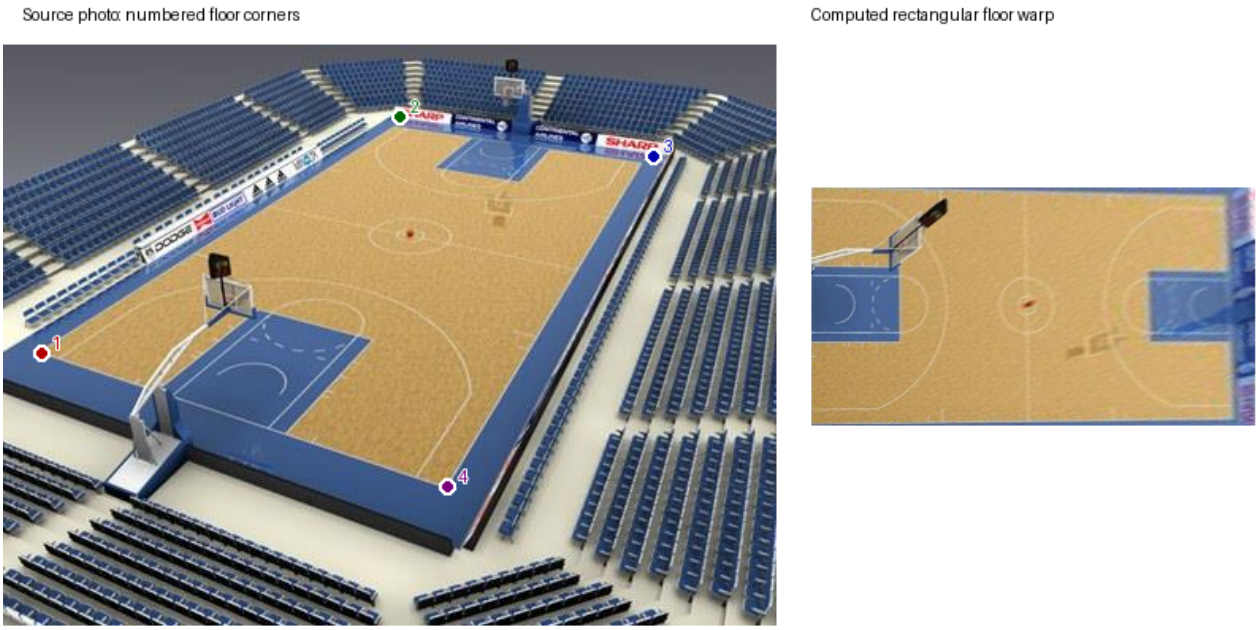


Fig. 2.1: Left: the lecture's court photograph, with four approximate floor corners added. Right: a computed rectangular warp of that region. The floor straightens; raised baskets and seating do not become a true overhead view. Source photograph: PDF page 71, printed slide 72; individual photographer uncredited. Warp and annotations are original.

2.3.1 One matrix, followed by a division

Formula — planar projective mapping

$$x' \sim Hx, \qquad H \in \mathbb{R}^{3\times 3}, \quad \det H \neq 0,$$
$$u' = \frac{h_{11}u + h_{12}v + h_{13}}{h_{31}u + h_{32}v + h_{33}}, \qquad v' = \frac{h_{21}u + h_{22}v + h_{23}}{h_{31}u + h_{32}v + h_{33}}. \tag{2.4}$$

$x = (u, v, 1)^T$ and $x' = (u', v', 1)^T$ are image or plane coordinates. Use consistent units within each

plane. The matrix scale follows the convention in (1.6).
The ordinary-coordinate fractions require a nonzero denominator. Zero means an ideal destination point. A single H models corresponding images of one plane, or a camera rotating about a fixed center. General camera translation with objects at different depths requires more than one homography.

Map a point or rectify a plane

1. Specify the direction: here H maps source coordinates to destination coordinates.
2. Multiply $H(u, v, 1)^T$ and divide by its last entry.
3. Check the result against known correspondences and image bounds.

To recover H, use at least four corresponding points in general position: no three collinear in either four-point set. See *estimation* and *pixel sampling*.

Worked example 2.2 — perspective changes spacing

Source: original small calculation for PDF pages 62–64.

Let $H = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1/2 & 0 & 1 \end{pmatrix}$. The source points $(0, 0)$, $(1, 0)$, $(2, 0)$ map to $(0, 0)$, $(2/3, 0)$, $(1, 0)$.

They stay on a line, but equal source intervals become unequal destination intervals: $2/3$ and $1/3$. The denominator depends on position. This is what an affine map cannot do along a line.

The finite point $(-2, 1)$ maps to $(-2, 1, 0)^T$, an ideal point. The homography remains invertible; only ordinary coordinates fail there.

The transformation families and their invariants are taught in *Move points without losing the geometry*. Here we use their projective case for an actual plane-to-image mapping.

2.3.2 A new detail: separate an affine map into rotations and scales

PDF page 68 adds a factorization of the affine linear part. It reveals the directions in which the map stretches the image.

Formula — directional scaling inside an affine map

$$A = R(\theta)R(-\phi)DR(\phi), \qquad D = \operatorname{diag}(s_1, s_2). \tag{2.5}$$

All matrices are 2×2; R is the planar rotation from (1.4). The angles orient the scaling axes and the final rotation. Nonzero signed scales allow any invertible real A, including orientation reversal. Translation is handled by the existing affine form in (1.6).

Read a directional stretch

1. Apply the rightmost rotation to express a vector in the scaling axes.

2. Scale the two components by s_1, s_2, then rotate back by $-\phi$.
3. Apply the final rotation by θ.

For $\theta = 0$, $\phi = 45°$, and $D = \text{diag}(2, 1)$, multiplication gives $A = \begin{pmatrix} 3/2 & -1/2 \\ -1/2 & 3/2 \end{pmatrix}$. It doubles the component along $(1, -1)^T$ and leaves the component along $(1, 1)^T$ unchanged. This is an original calculation of the slide's added factorization.

2.3.3 Cross-ratio: a ratio perspective cannot change

Ordinary distances change in perspective, but a particular combination of four collinear locations survives.

Formula — cross-ratio of four collinear points

$$\text{CR}(a, b; c, d) = \frac{(c - a)(d - b)}{(c - b)(d - a)}. \tag{2.6}$$

a, b, c, d are distinct scalar positions along one line, in a common coordinate. The result is unitless and depends on the chosen ordering. This finite-coordinate expression assumes nonzero denominators; ideal points are handled by limits or homogeneous coordinates. An invertible projective mapping of the line preserves the result.

Check a projective invariant

1. Keep the same ordering of four distinct collinear points before and after mapping.
2. Compute the two cross-ratios and compare.
3. Do not substitute ordinary image distances for directed line coordinates.

For $a, b, c, d = 0, 1, 2, 3$, the result is $4/3$. Under Example 2.2's map, the positions become $0, 2/3, 1, 6/5$; substitution still gives $4/3$.

2.3.4 What is missing from the court assignment?

We need **correspondences and a target shape**. Select four floor corners in cyclic order, match them to a rectangle, then estimate H. Its width-to-height ratio must come from known court geometry if we want metric proportions. An arbitrary rectangle can look tidy while having the wrong scale ratio.

Our figure uses approximate source pixels $(24, 194)$, $(250, 45)$, $(410, 70)$, $(280, 278)$, matched to $(0, 0)$, $(279, 0)$, $(279, 149)$, $(0, 149)$. The chosen aspect ratio is illustrative, not a measurement of the court. Boundary pixels are approximate because the supplied photo is small. **A floor homography cannot reveal hidden floor or flatten raised objects correctly.**

2.4 Image warping: where do the new pixels come from?

Mapping a point is not enough to create an image. Source pixels lie on a grid; transformed positions usually fall between destination pixels. We need a rule for filling the destination grid.

Source: PDF pages 72–81, printed slides 73–82; the section credits Steve Seitz.

2.4.1 Move colors, or look them up?

Image filtering changes pixel values at fixed coordinates, such as darkening a picture. **Image warping** changes where the content appears. A global parametric warp uses the same coordinate function everywhere, though its displacement can vary with position.

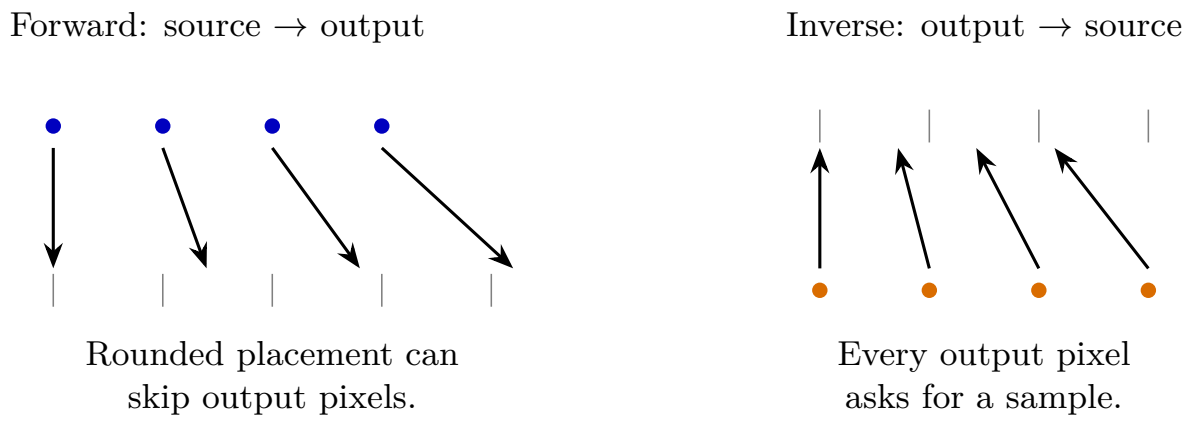


Fig. 2.2: Forward mapping asks where each source pixel lands. Inverse mapping asks where each output pixel should sample. Original diagram of the distinction on PDF pages 76–79.

Formula — inverse image sampling

$$g(u', v') = f(\text{dehom}(H^{-1}(u', v', 1)^T)) \,. \tag{2.7}$$

f is the source image, g the output; H maps **source to destination**. The operator dehom divides the first two coordinates by the last. Pixel values may be grayscale scalars or color vectors. This describes continuous sampling; discrete images require interpolation and a boundary rule.

Fill an output image

1. Visit every output pixel center and multiply by H^{-1}.
2. Divide by the homogeneous denominator; mark invalid/out-of-source samples as background.
3. Interpolate the source image and store that value.

This avoids holes caused by rounded forward placement. Forward warping is still useful when samples must

be projected or splatted. When shrinking an image substantially, prefilter it to reduce aliasing; interpolation alone is insufficient.

2.4.2 Bilinear interpolation: blend four neighbors

Suppose inverse mapping returns source location $(i + a, j + b)$, where i, j are integers and $0 \le a, b < 1$. First interpolate horizontally in each row, then vertically between those results.

Formula — bilinear interpolation

$$f(i+a, j+b) \approx (1-a)(1-b)f(i,j) + a(1-b)f(i+1,j) + (1-a)bf(i,j+1) + abf(i+1,j+1). \tag{2.8}$$

a, b are unitless fractions of a pixel; the output has the same intensity units as the samples. We write coordinates as (column, row); an image array usually indexes $f[\text{row}, \text{column}]$. The four weights are nonnegative and sum to one. Specify how samples outside the image are handled.

Calculate a sampled intensity

1. Take the floor of each continuous coordinate to find i, j.
2. Compute a, b and the four weights.
3. Blend each image channel separately.

Check that a constant patch remains constant and that integer coordinates reproduce the stored pixel. See Example 2.3.

Worked example 2.3 — a pixel between pixels

Source: original numerical companion to PDF page 80, printed slide 81.
The four intensities at $(i, j), (i+1, j), (i, j+1), (i+1, j+1)$ are $10, 30, 50, 70$. At $a = 1/4$, $b = 1/2$, the weights are $3/8, 1/8, 3/8, 1/8$.
The sampled value is $(3 \cdot 10 + 30 + 3 \cdot 50 + 70)/8 = 35$. Equivalently, the two horizontal interpolations give 15 and 55, whose halfway blend is 35. It lies within the range of the four inputs.

Check 2.2 — which inverse?

If H shifts source content ten pixels right, where does output pixel $(20, 8)$ sample?

Answer

At source $(10, 8)$. Using H instead of H^{-1} would sample $(30, 8)$ and move content in the wrong direction.

2.5 Estimate a reliable mapping

Four matches can determine a homography, but real matches are noisy and some are wrong. **DLT** solves for the matrix; **RANSAC** finds matches that agree with one matrix. They solve different problems.

Source: PDF pages 82–104, printed slides 83–105. These sections credit R. Hartley, A. Zisserman, and M. Pollefeys. Lecture 3 will add the assigned homography/probability quizzes and further numerical practice.

2.5.1 DLT: turn matches into linear equations

The **direct linear transformation (DLT)** starts from $x' \sim Hx$. Their cross product is zero because the vectors represent the same projective point: $x' \times Hx = 0$.

For finite matches $(u, v) \leftrightarrow (u', v')$, multiply out the denominators in (2.4). Each match supplies two independent equations in the nine entries of H.

Formula — homography DLT system

$$
\begin{aligned}
A_i &= \begin{pmatrix} u & v & 1 & 0 & 0 & 0 & -u'u & -u'v & -u' \\ 0 & 0 & 0 & u & v & 1 & -v'u & -v'v & -v' \end{pmatrix}, \\
h &= (h_{11}, h_{12}, h_{13}, h_{21}, h_{22}, h_{23}, h_{31}, h_{32}, h_{33})^T, \\
A &= \begin{pmatrix} A_1 \\ \vdots \\ A_n \end{pmatrix}, \qquad \min_{\|h\|=1} \|Ah\|^2.
\end{aligned}
\tag{2.9}
$$

A is $2n \times 9$, h is 9×1. The constraint excludes the useless zero solution. Four nondegenerate exact matches give rank eight and a one-dimensional nullspace; with noisy extra matches, minimize the residual instead of expecting exact zero.
The finite-coordinate rows assume each destination's last coordinate is nonzero. For ideal points, retain appropriate independent rows of the full cross-product system. Three cross-product equations have only two independent constraints; dropping the same row blindly can lose information.

Estimate a homography with SVD

1. Normalize coordinates as described below; reject a degenerate four-point sample.
2. Stack two rows per match to form A.
3. Compute $A = U\Sigma V^T$ using singular value decomposition. Choose the **right singular vector** with smallest singular value and reshape it row by row into H.
4. Undo coordinate normalization and check projected matches.

For an 8×9 system, request the full right-singular-vector basis: a reduced SVD can omit the ninth vector containing the nullspace. Do not confuse the last row of V^T with the last column of V^T.

Fixing $h_{33} = 1$ is a tempting shortcut, but valid homographies can have $h_{33} = 0$. Unit-vector normalization avoids privileging that entry. DLT minimizes **algebraic error**, which depends on coordinate scaling. Pixel reprojection error is a more interpretable final accuracy measure.

Worked example 2.4 — recover a simple planar map

Source: original numerical companion to PDF pages 83–94.

Match $(0,0),(1,0),(1,1),(0,1)$ to $(2,1),(4,1),(4,4),(2,4)$ in that order. The first correspondence produces rows

$$(0,0,1,0,0,0,0,0,-2), \qquad (0,0,0,0,0,1,0,0,-1).$$

For $(1,0) \leftrightarrow (4,1)$ the rows are

$$(1,0,1,0,0,0,-4,0,-4), \qquad (0,0,0,1,0,1,-1,0,-1).$$

The other two matches fill the remaining four rows. The resulting matrix has rank eight. Its null vector, reshaped and rescaled, gives $H = \begin{pmatrix} 2 & 0 & 2 \\ 0 & 3 & 1 \\ 0 & 0 & 1 \end{pmatrix}$.

Check the unmeasured center: $H(1/2,1/2,1)^T = (3,5/2,1)^T$, the center of the destination rectangle. All four measured corners reproject exactly. Three collinear source points would invalidate this four-point recovery.

2.5.2 Normalize: put the coordinates on a sensible scale

Raw pixel coordinates may be hundreds, while their products in A reach tens of thousands. **Normalized DLT** recenters and rescales each image's points independently, reducing this imbalance.

Formula — coordinate normalization and recovery

$$\begin{aligned} c &= \frac{1}{n}\sum_i p_i, \quad \bar{r} = \frac{1}{n}\sum_i \|p_i - c\|, \quad s = \frac{\sqrt{2}}{\bar{r}}, \\ T &= \begin{pmatrix} s & 0 & -sc_u \\ 0 & s & -sc_v \\ 0 & 0 & 1 \end{pmatrix}, \qquad \tilde{x} = Tx, \quad \tilde{x}' = T'x', \\ H &= (T')^{-1}\widetilde{H}T. \end{aligned} \tag{2.10}$$

$p_i = (u_i, v_i)^T$ are finite 2D points; T, T', H are 3×3. The mean distance from the normalized centroid becomes $\sqrt{2}$. Require $\bar{r} > 0$; coincident points provide no spatial information. Normalization improves conditioning, not incorrect matches or degeneracy.

Use normalized DLT

1. Compute separate T, T' from the source and destination matches.
2. Fit $\widetilde{H}$ to their normalized coordinates using (2.9).
3. Recover $H = (T')^{-1}\widetilde{H}T$ and evaluate error in original coordinates.

The source square in Example 2.4 has centroid $(1/2,1/2)$, mean radius $1/\sqrt{2}$, and scale $s = 2$. Its normalized corners are $(\pm 1, \pm 1)$, at radius $\sqrt{2}$.

Why this recovery order? The normalized mapping is $T'x' \sim \widetilde{H}Tx$. Multiply by $(T')^{-1}$ to isolate the original destination coordinates. PDF page 92 also shows a simpler image-size scaling; it is an alternative heuristic, not the same transformation as centroid/mean-distance normalization.

2.5.3 RANSAC: do not let wrong matches vote as measurements

An ***inlier*** agrees with the model within a chosen error threshold. An **outlier** does not. ***RANSAC*** means random sample consensus: fit small random subsets, then count how many matches support each hypothesis.

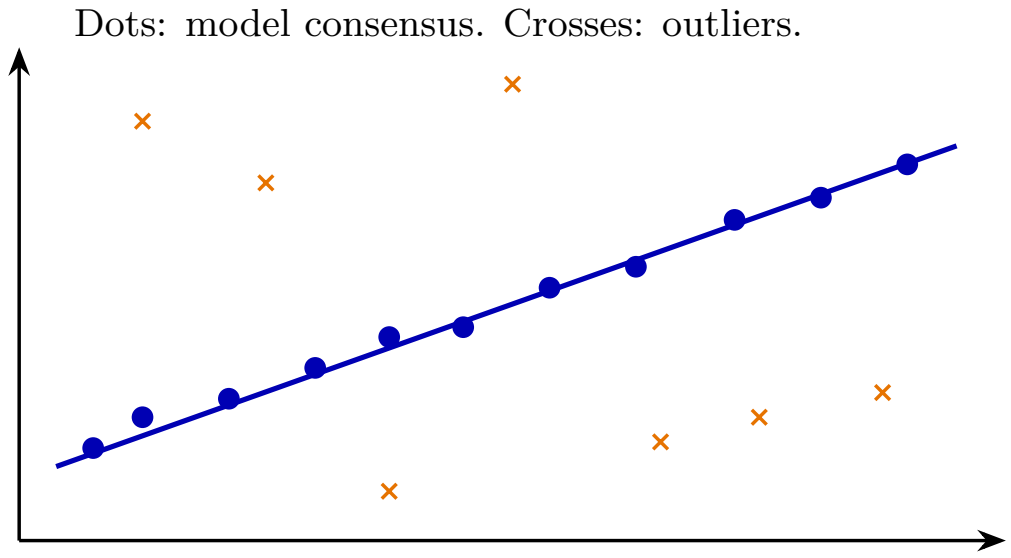


Fig. 2.3: The idea behind the lecture's robust-line example: a large consistent group supports the correct model despite scattered bad measurements. Original schematic; homography hypotheses use four matches rather than two line points.

Formula — RANSAC trial count and inlier score

$$N = \left\lceil \frac{\log(1-p)}{\log(1-w^s)} \right\rceil, \qquad r_i = \|p'_i - \mathrm{dehom}(Hx_i)\|_2, \qquad n_{\text{in}} = \#\{i : r_i < \tau\}. \tag{2.11}$$

p is the desired probability of at least one all-inlier sample, $w = 1 - e$ the inlier fraction, and s the sample size (four for a homography). Here residuals and threshold τ are in destination pixels. N is a dimensionless integer.

The usual w^s model assumes independent draws, or a large pool approximating them, and nondegenerate all-inlier samples. Trials are independent. For sampling s distinct matches from a small pool of n containing I true inliers, the exact all-inlier probability is $\binom{I}{s}/\binom{n}{s}$. Neither probability guarantees the selected model is correct.

The derivation is short: a trial is all-inlier with probability w^s. All N trials fail with probability $(1 - w^s)^N$. Set that failure probability at most $1 - p$, take logarithms, and round up.

Fit a homography robustly

1. Sample four matches; reject collinear or otherwise degenerate configurations; estimate H.
2. Score every match against H using a fixed error definition and threshold.
3. Keep the best consensus and repeat to the trial budget. Update the budget from a better observed inlier ratio, while recognizing that it is an estimate.
4. Refit to the best inliers, then refine pixel reprojection error if needed.

For $w = 0$ no finite budget succeeds; for $w = 1$ one nondegenerate trial suffices. A large consensus of repeated patterns can still support the wrong alignment. See Example 2.5.

Worked example 2.5 — why a better inlier fraction saves work

Source: numerical expansion of PDF page 98's trial table; application figures from PDF page 104.
With $p = 0.99$, $w = 0.5$, and $s = 4$, the continuous budget is $\log(0.01)/\log(15/16) \approx 71.36$. Round **up** to 72 trials. At $w = 0.8$, it falls to 9.
The lecture's building-image example starts with 268 putative matches. A 151-match consensus corresponds to roughly 56% inliers; using that rounded ratio gives about 45 trials. The slide reports 43 iterations: the simple probability model is a budget estimate, not a reproduction of the implementation's exact stopping history. Guided matching later finds 262 final inliers; this is a new matching stage, not simply the original consensus.

An expected consensus threshold is approximately wn (rounded as needed), but it is not a statistical guarantee. **LMedS**, least median of squares, instead minimizes median squared residual; it is a different robust objective. Ordinary least squares can be dominated by gross outliers. Iterative refinement is useful **after** robust selection; iteration alone does not make an estimator robust.

2.5.4 The complete slide pipeline

Detect interest points → propose matches → robustly estimate H → refine using inliers → use H to guide more matches. The slides name sum of absolute differences (SAD), sum of squared differences (SSD), and zero-mean normalized cross-correlation (ZNCC) as patch comparisons. SAD/SSD compare pixel intensities directly; ZNCC compares mean-centered, normalized patterns and needs nonzero patch variance. Lecture 5 develops matching and exposure changes.

When motion is known to be small, restrict candidate locations; do not impose that restriction on arbitrary viewpoint changes. Nonlinear refinement may use Levenberg–Marquardt to minimize an image-error objective, rather than the initial algebraic DLT objective.

Check 2.3 — can more points rescue a bad model?

Would fitting one homography to hundreds of matches on several depths always work better than using four floor corners?

Answer

No. Extra measurements cannot repair the wrong geometric model. With translation, a single homography generally fits one plane, not objects at many depths.

2.6 Distortion and two-view geometry

A planar warp is powerful, but it cannot explain curved lens distortion or general 3D parallax. The final slides introduce the tools for those cases.

Source: PDF pages 105–129, printed slides 106–130; the section credits R. Hartley, A. Zisserman, and M. Pollefeys. The assigned epipolar/stereo quizzes and a fuller estimation treatment belong to Lecture 4.

2.6.1 Straight lines first: correct lens distortion

Radial distortion moves points along directions from a distortion center. Straight scene edges can become curved. A homography preserves lines, so it cannot generally undo this bending.

Formula — radial correction model

$$r^2 = (u - c_u)^2 + (v - c_v)^2, \qquad \begin{pmatrix} \hat{u} \\ \hat{v} \end{pmatrix} = \begin{pmatrix} c_u \\ c_v \end{pmatrix} + (1 + k_1 r^2 + k_2 r^4 + \cdots) \begin{pmatrix} u - c_u \\ v - c_v \end{pmatrix}. \tag{2.12}$$

Here (u, v) are measured distorted coordinates and hats denote corrected coordinates. The coefficients belong to **this correction direction**; a forward-distortion model generally needs different coefficients or numerical inversion. Use calibrated units: if r is in pixels, k_1 has units pixel^{-2} and k_2 pixel^{-4}; normalized coordinates are also common.
This low-order model describes radial effects, not every lens defect. The distortion center may differ from the principal point. PDF page 109 mixes radius and squared-radius notation; writing r^2 explicitly avoids that ambiguity.

Correct a point before geometric estimation

1. Use the calibrated center, units, coefficients, and mapping direction.
2. Compute r^2 and the radial factor, then correct the coordinate.
3. Check that straight scene edges become straighter; resample images with an appropriate inverse map.

For center $(0, 0)$, point $(100, 0)$ pixels, $k_1 = 10^{-6}$ pixel^{-2}, and no higher terms, the factor is 1.01 and the corrected point is $(101, 0)$. A positive coefficient here expands radii; never transfer that sign interpretation to the inverse convention blindly.

The slides suggest estimating distortion jointly with other camera parameters or by straightening known straight edges. Their room-image before/after sequence illustrates why this matters: ideal projective geometry should be applied to undistorted coordinates.

2.6.2 One point becomes a line in the other image

With two camera centers C, C' and a scene point X, the three define an **epipolar plane**. It cuts each image plane in an ***epipolar line***. If the first image point is known but its depth is not, its possible matches lie along that line in the second image.

The ***epipole*** is the image of the other camera center; every epipolar line in an image passes through its epipole. The line between camera centers is the **baseline**.

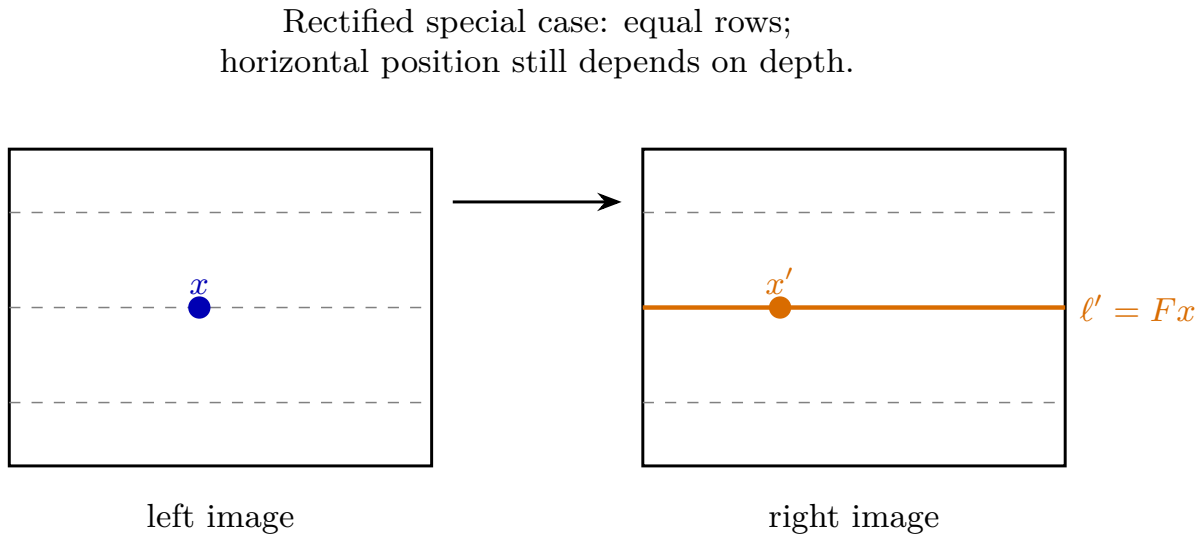


Fig. 2.4: A special, rectified case: the right match must lie on the corresponding row. Its location along that row still depends on depth. Original schematic developing PDF pages 113–119.

2.6.3 The fundamental matrix maps points to lines

Formula — epipolar constraint and epipoles

$$\begin{aligned} \ell' &= Fx, \qquad \ell = F^T x', \qquad x'^T F x = 0, \\ Fe &= 0, \qquad F^T e' = 0, \qquad \operatorname{rank} F = 2. \end{aligned} \tag{2.13}$$

x, x' are homogeneous image 3×1 points; ℓ, ℓ' are line coefficients, e, e' epipoles. F is a 3×3 ***fundamental matrix***, defined up to nonzero scale. It has seven DOF: nine entries minus scale minus the rank-two constraint. Reversing the image order transposes F.
This ideal constraint assumes corresponding undistorted points and distinct camera centers. At an epipole, the corresponding line vector can vanish. Pure rotation has no nonzero baseline and does not determine a nonzero fundamental matrix from the camera pair. The equation constrains correspondence; it does not identify a unique match.

Turn a point into a correspondence constraint

1. Multiply F by the first image point to obtain the second image's line.
2. Test a candidate match by its dot product with that line.
3. For noisy data, use a scale-normalized geometric error instead of comparing raw $x'^T F x$ values across arbitrary scalings.

Check matrix rank and image order. See Example 2.6.

Worked example 2.6 — a horizontal epipolar line

Source: original numerical companion to the parallel-camera example, PDF page 119.

Let $F = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix}$ and $x = (2, 3, 1)^T$. Then $Fx = (0, -1, 3)^T$: candidate matches satisfy $-v' + 3 = 0$.

Both $(5, 3, 1)^T$ and $(8, 3, 1)^T$ satisfy the constraint; $(5, 4, 1)^T$ does not. The two valid candidates show why an epipolar line is not a point prediction. The matrix has rank two and both epipoles are $(1, 0, 0)^T$, the horizontal ideal direction.

2.6.4 Where does the matrix come from?

The source derives F by projecting two points on the first camera's viewing ray into the second image. Their join is the epipolar line.

Formula — construct F from two camera matrices

$$PC = 0, \qquad e' = P'C, \qquad F = [e']_\times P' P^+. \tag{2.14}$$

P, P' are rank-three 3×4 camera matrices; C is the first camera center as a homogeneous 4-vector; P^+ is a 4×3 right pseudoinverse with $PP^+ = I_3$. The matrix $[e']_\times$ takes a cross product with e'.

For distinct centers, $P^+ x$ is one 3D point projecting to x; adding λC stays on its projective ray. Its second image lies on the line joining $P'P^+x$ to $P'C$, producing the formula. Coincident centers give $P'C = 0$, so this construction gives zero, not a valid rank-two F.

Check a camera-derived epipolar geometry

1. Find a nonzero null vector C of P and compute $e' = P'C$.
2. Form the pseudoinverse and cross-product matrix, then multiply in the stated order.
3. Project a test 3D point in both cameras and verify the scalar constraint.

The cameras $P = [I \mid 0]$, $P' = [I \mid (-1, 0, 0)^T]$ give Example 2.6's matrix up to sign. A scene point $(2, 3, 1, 1)^T$ projects to $(2, 3, 1)^T$ and $(1, 3, 1)^T$, on the same row.

For a fixed nondegenerate camera pair, F is unique **up to scale**. Estimating it from a finite set of matches can be ambiguous, particularly with planar scene points.

Formula — linear equations for F from matches

$$\begin{pmatrix} u'u & u'v & u' & v'u & v'v & v' & u & v & 1 \end{pmatrix} f = 0, \qquad Af = 0. \tag{2.15}$$

$f = (F_{11}, F_{12}, F_{13}, F_{21}, \ldots, F_{33})^T$ is 9×1; one finite match contributes one row. At least eight suitably independent matches give the linear estimate. This differs from homography DLT's **two** equations per match. Noisy linear solutions need a rank-two correction.

Recognize the eight-point estimation route

1. Normalize both sets of image coordinates and stack the rows.
2. Find the smallest right singular vector and reshape it into a matrix.
3. Enforce rank two by setting its smallest singular value to zero.
4. Undo normalization: if normalized points are $Tx, T'x'$, then $F = T'^T \widetilde{F} T$; verify on held-out matches.

For Example 2.6 and match $(u, v) = (2, 3)$, $(u', v') = (5, 3)$, the row is $(10, 15, 5, 6, 9, 3, 2, 3, 1)$. Dotting it with the row-stacked F gives $-3 + 3 = 0$. Lecture 4 will develop noisy estimation and its assigned quiz calculations.

2.6.5 Motion patterns and rectification

The three slide examples show different locations for the epipoles:

- **Converging cameras:** epipolar lines fan toward finite epipoles, possibly outside the images.
- **Translation parallel to the image plane:** epipoles are ideal; for a horizontal baseline and aligned cameras, epipolar lines are horizontal.
- **Forward translation:** epipoles can lie inside the images, with radial epipolar lines. Ordinary rectangular planar rectification becomes problematic near that epipole.

Rectification warps two views so corresponding epipolar lines become matching horizontal rows. It simplifies the search from two dimensions to one; it does not eliminate matching ambiguity or depth variation.

Formula — rectified coordinates and line geometry

$$\bar{x} \sim Hx, \quad \bar{x}' \sim H'x', \quad \bar{F} = H'^{-T} F H^{-1}, \qquad He \sim H'e' \sim (1, 0, 0)^T. \tag{2.16}$$

H, H' are nonsingular 3×3 image warps. Moving both epipoles to the horizontal ideal point makes epipolar lines horizontal; the two warps must also align the corresponding row coordinates. A finite epipole is sent to infinity, so its neighborhood cannot retain a finite undistorted rectangular image.

Interpret a rectified pair

1. Estimate camera/epipolar geometry and choose compatible rectifying warps.

2. Resample both images by inverse warping.
3. Check that known matches share the same row; then search along that row.

Example 2.6 is already rectified: identity warps preserve its horizontal lines and equal-row constraint. The final slide's paired objects illustrate the visual goal. Detailed stereo depth follows in Lecture 4.

2.7 Review and formula checklist

Check the new tools below. Foundation explanations and Spring 2020 Quiz 1 solutions are in *Foundation quiz practice*. Source ranges below are PDF pages in `cs532_f25_Week2.pdf`.

Topic and source pages	Canonical formula or explanation	Practice
Quaternion conversion, 44–49	(2.1); (2.2)	*Example 2.1*
Smooth rotation interpolation, 45	(2.3) (explanatory companion)	Identity to quarter-turn midpoint
Directional affine scaling, 68	(2.5)	Stretch along a diagonal
Planar rectification and cross-ratio, 62–71	(2.4); (2.6)	*Example 2.2*; court warp
Warping and interpolation, 72–81	(2.7); (2.8)	*Example 2.3*
Homography estimation, 82–94	(2.9); (2.10)	*Example 2.4*
Robust estimation and matching, 95–104	(2.11); *Estimate a reliable mapping*	*Example 2.5*
Radial correction, 105–110	(2.12)	Correct $(100, 0)$ to $(101, 0)$
Epipolar geometry, 111–126	(2.13); (2.14); (2.15)	*Example 2.6*
Rectification, 127–129	(2.16)	Equal-row and epipole checks

2.7.1 Can you do these without notes?

- Convert a rotation matrix to a quaternion and verify the round trip.
- Check quaternion signs with a cyclic-axis rotation and handle a half-turn without dividing by zero.
- Explain why a homography has eight DOF and needs four nondegenerate matches.
- Write two DLT rows for a match and identify the correct right singular vector.
- Undo coordinate normalization in the correct order.
- Explain why inverse warping still needs interpolation and a boundary rule.
- Distinguish noise, outliers, and the wrong geometric model.
- Map an image point to an epipolar line, then test a possible match.

Choose the right image model: a homography maps planar points to points; a fundamental matrix maps a point to a correspondence line in another view.

Finish with the *TLDR and short quiz*. Lecture 3 will expand camera calibration and homography practice; Lecture 4 will expand epipolar estimation and stereo. Those later quiz assignments remain in their planned chapters.

2.8 TLDR

Lecture 2 in one sentence: estimate planar image mappings from trustworthy matches, resample them correctly, and recognize when distortion or 3D depth requires another model.

2.8.1 The important things

1. **Convert quaternions consistently.** This chapter adds quaternion-to-matrix and matrix-to-quaternion formulas. The scalar extraction is $w = \frac{1}{2}\sqrt{1 + \operatorname{tr} R}$; near a half-turn, use a largest-component branch. See *Quaternions: convert without changing the rotation*.
2. **SLERP interpolates orientations.** Align endpoint quaternion signs, then interpolate along the unit sphere. For nearly identical orientations, normalized linear interpolation avoids an unstable denominator.
3. **A plane can be rectified with four nondegenerate matches.** A homography supplies the mapping; the destination rectangle's aspect ratio needs known geometry for metric accuracy. Raised objects and hidden floor cannot be reconstructed by a floor warp. See *Homographies: straighten a photographed plane*.
4. **Cross-ratio survives perspective.** For four ordered collinear coordinates, $\mathrm{CR}(a, b; c, d) = (c - a)(d - b)/((c - b)(d - a))$. Ordinary spacings may change while this value stays fixed.
5. **Inverse warping fills the output grid.** For each destination pixel, use H^{-1} to locate its source sample. Bilinear interpolation blends four neighbors with weights summing to one; strong shrinking also needs antialiasing. See *Image warping: where do the new pixels come from?*.
6. **DLT estimates the mapping; normalization stabilizes it.** Each homography match contributes two rows to a $2n \times 9$ system. The smallest right singular vector supplies the matrix entries. Normalize each image independently, then recover $H = (T')^{-1}\widetilde{H}T$. See *Estimate a reliable mapping*.
7. **RANSAC rejects inconsistent matches.** Sample → fit → score → repeat → refit. Under its sampling assumptions, use $N = \lceil \log(1 - p)/\log(1 - w^s) \rceil$, where w is the inlier fraction, s the sample size, and p the desired sampling success probability. Consensus is not proof of a correct model.
8. **Distortion and depth variation require different tools.** Radial correction straightens lens-bent edges. The fundamental matrix constrains correspondence by $\ell' = Fx$ and $x'^T Fx = 0$; it has rank two and seven DOF. Rectification aligns corresponding epipolar lines with matching rows. See *Distortion and two-view geometry*.

2.8.2 Short quiz

These original questions test the additions in this chapter. Foundation quiz solutions are in *Foundation quiz practice*.

1. For a proper rotation with trace zero, what scalar quaternion component does the trace branch give?
2. Before SLERP, what should you do if the endpoint quaternions have a negative dot product?
3. What is $\mathrm{CR}(0, 1; 2, 3)$? Should a projective warp of the line change it?
4. How many nondegenerate matches and how many DLT rows are needed for a minimal homography estimate?
5. A source-to-destination warp translates content by $(6, -2)$ pixels. Where does destination pixel $(10, 10)$ sample?
6. Four neighboring intensities are $0, 20, 40, 60$. What is their bilinear value at the center of the square?
7. For $p = 0.99$, $w = 0.8$, $s = 4$, is the standard RANSAC budget 8 or 9 trials? Does it guarantee the correct model?
8. If $Fx = (0, -1, 4)^T$, what must a matching ordinary point (u', v') satisfy? Is u' determined?

2.8.3 Answers

Show answers and short explanations

1. $w = 1/2$**.** Substitute the trace into $\frac{1}{2}\sqrt{1 + \operatorname{tr} R}$; the chosen branch takes the nonnegative sign.
2. **Negate one endpoint.** This aligns quaternion signs so interpolation follows a shortest rotation path.
3. $4/3$**, unchanged.** Substitute $(2-0)(3-1)/((2-1)(3-0))$. Preserve the ordering of the four points.
4. **Four matches, eight rows.** Each match gives two independent equations; the exact nondegenerate system is 8×9 with rank eight.
5. **Source** $(4, 12)$**.** Undo the translation: subtract six from the column and add two to the row.
6. **30.** At the center all four weights are $1/4$.
7. **Nine.** Round the continuous estimate, about 8.74, up. It gives a sampling probability under stated assumptions, not a correctness guarantee.
8. $v' = 4$**;** u' **remains free.** The line equation is $-v' + 4 = 0$; image content must still identify a match along it.

CHAPTER

THREE

LECTURE 3: CAMERA PROJECTION, HOMOGRAPHIES, AND RANSAC

> **Note**
>
> Planning outline — this lecture has not yet been written. Its full slide audit, examples, and formulas will be completed in lecture order.

3.1 Planned scope

Image formation; camera parameters; homographies; estimation and robust fitting.

Source deck: `wk3_cs532s26_ImageFormation+Homographies-1.pdf`. Full explanations will record verified printed slide labels and one-based PDF page numbers separately.

3.2 Examples and practice

Select the most useful examples from the actual slides and integrate the assigned questions from the *quiz index*. Missing quiz coverage will be filled with new, clearly labeled worked examples.

3.3 Completion checklist

- Audit every substantive slide topic and formula.
- Write explanations, derivations, and formula/application boxes.
- Add selected slide illustrations and independently checked quiz examples.
- Complete keywords, glossary entries, and source references.
- Check both the website and cumulative printable edition.

CHAPTER

FOUR

LECTURE 4: TWO-VIEW GEOMETRY AND STEREO

> **Note**
>
> Planning outline — this lecture has not yet been written. Its full slide audit, examples, and formulas will be completed in lecture order.

4.1 Planned scope

Distortion; epipolar geometry; fundamental and essential matrices; stereo.

Source deck: `wk4_cs532s26_TwoViewGeometry.pdf`. Full explanations will record verified printed slide labels and one-based PDF page numbers separately.

4.2 Examples and practice

Select the most useful examples from the actual slides and integrate the assigned questions from the *quiz index*. Missing quiz coverage will be filled with new, clearly labeled worked examples.

4.3 Completion checklist

- Audit every substantive slide topic and formula.
- Write explanations, derivations, and formula/application boxes.
- Add selected slide illustrations and independently checked quiz examples.
- Complete keywords, glossary entries, and source references.
- Check both the website and cumulative printable edition.

CHAPTER

FIVE

LECTURE 5: CORNERS, TRACKING, AND MATCHING

> **Note**
>
> Planning outline — this lecture has not yet been written. Its full slide audit, examples, and formulas will be completed in lecture order.

5.1 Planned scope

Corner detection; feature tracking; descriptors and matching.

Source deck: `wk5_cs532_Corners.pdf`. Full explanations will record verified printed slide labels and one-based PDF page numbers separately.

5.2 Examples and practice

Select the most useful examples from the actual slides and integrate the assigned questions from the *quiz index*. Missing quiz coverage will be filled with new, clearly labeled worked examples.

5.3 Completion checklist

- Audit every substantive slide topic and formula.
- Write explanations, derivations, and formula/application boxes.
- Add selected slide illustrations and independently checked quiz examples.
- Complete keywords, glossary entries, and source references.
- Check both the website and cumulative printable edition.

CHAPTER

SIX

LECTURE 6: STRUCTURE FROM MOTION

> **Note**
>
> Planning outline — this lecture has not yet been written. Its full slide audit, examples, and formulas will be completed in lecture order.

6.1 Planned scope

Reconstruction ambiguity; factorization; triangulation; bundle adjustment.

Source deck: `wk6_cs532s24_SFM1.pdf`. Full explanations will record verified printed slide labels and one-based PDF page numbers separately.

6.2 Examples and practice

Select the most useful examples from the actual slides and integrate the assigned questions from the *quiz index*. Missing quiz coverage will be filled with new, clearly labeled worked examples.

6.3 Completion checklist

- Audit every substantive slide topic and formula.
- Write explanations, derivations, and formula/application boxes.
- Add selected slide illustrations and independently checked quiz examples.
- Complete keywords, glossary entries, and source references.
- Check both the website and cumulative printable edition.

Study and reference

This reference will grow one lecture at a time. Every entry points to a named formula box, its assumptions, derivation, and application procedure. Lectures 1 and 2 are populated; Lectures 3–6 remain pending.

Foundation	Formula	Explanation and application
Homogeneous scale	(1)	*Notation and coordinate conventions*

Lecture 1

See the *Lecture 1 formula and source checklist* for the full inventory. Each entry links to its formula box, assumptions, and companion application procedure.

Lecture 2

See the *Lecture 2 formula and source checklist* for quaternion conversions, directional scaling, homographies, image sampling, DLT/normalization, RANSAC, radial correction, and epipolar constraints. Repeated Lecture 1 formulas retain their original canonical boxes.

QUIZ PRACTICE INDEX

Placements follow `quizByLec.md`. Lecture 1 now includes independently checked worked examples and a supplementary depth-map bridge. Spring 2020 Quiz 1 retains its Lecture 2 assignment but its solutions are in Lecture 1 to avoid repeating foundations; Lectures 3–6 remain planned. Duplicate sources are retained for later closed-notes retests.

Lecture 1

Completed reading links: *Example 1.10* (relative camera pose), *Example 1.11* (camera/object plane), *Example 1.12* (face normals), and *Example 1.13* (rotated normals). The *supplementary depth-map solution* follows the camera explanation. Fall 2025 Quiz 1 Q1–Q2 remains deferred to Lecture 3.

Fall 2025 Quiz 1

Source: `cs532f25_quiz_1_solution.pdf`

Planned use: Relative poses; plane construction; probability (secondary L3).

Fall 2017 Quiz 9

Source: `cs532_f17_quiz9.pdf`

Planned use: Rotated normals; depth backprojection (extension after L3/4).

Lecture 2

Spring 2020 Quiz 1

Source: `cs532s20_quiz1.pdf`

Teaching location: *Foundation quiz practice*, alongside the concepts it tests.

Worked solutions: *Example 1.17* (P1), *Example 1.18* (P2.1–2), and *Example 1.19* (P3.1–2). All parts include assumptions and degeneracy checks.

Lecture 3

Spring 2019 Quiz 3

Source: `cs532s19-quiz3.pdf`

Planned use: Pentagon matches; homography; RANSAC; symmetry.

Spring 2019 Quiz 2

Source: `cs532s19_quiz2-2.pdf`

Planned use: FOV; intrinsics; projection; principal point.

Spring 2020 Quiz 3 (mixed printed header)

Source: `cs532s20_quiz3.pdf`

Planned use: Camera calibration retest.

Retest: near-duplicate of cs532s19_quiz2-2.pdf.

Spring 2020 Quiz 2

Source: `cs532s20_quiz2.pdf`

Planned use: Pinhole projection; ray cross product.

Filename 2015 Quiz 9; printed Spring 2019

Source: `cs532f15_quiz9.pdf`

Planned use: Ray-plane depth.

Lecture 4

Filename 2015 Quiz 4; printed Spring 2019

Source: `cs532f15_quiz4.pdf`

Planned use: Fundamental matrix; epipolar constraint.

Spring 2019 Quiz 4

Source: `cs532s19_quiz4.pdf`

Planned use: Epipolar geometry retest.

Retest: duplicate of cs532f15_quiz4.pdf per study notes.

Fall 2025 Quiz 2

Source: `cs532f25_quiz_2_solution.pdf`

Planned use: Three-view lines; epipolar-consistent points; source erratum.

Fall 2025 Quiz 3

Source: `cs532f25_quiz_3_solution.pdf`

Planned use: Stereo depth; error; disparity bounds.

Spring 2019 Quiz 5

Source: `cs532s19_quiz5-2.pdf`

Planned use: Exact positive/negative disparity error.

Spring 2020 Quiz 5

Source: `cs532s20_quiz5.pdf`

Planned use: Disparity error retest.

Retest: duplicate of cs532s19_quiz5-2.pdf per study notes.

Lecture 5

Fall 2025 Quiz 4

Source: `cs532f25_quiz_4_solution.pdf`

Planned use: KLT; exposure normalization.

Spring 2019 Quiz 6

Source: `cs532s19_quiz6-2.pdf`

Planned use: Corners; edges; isolated pixels; aperture problem.

Lecture 6

Fall 2025 Quiz 5

Source: `cs532f25_quiz_5_solution-1.pdf`

Planned use: Reconstruction upgrades; factorization vs sequential.

Fall 2025 Quiz 6

Source: `cs532f25_quiz_6_solution.pdf`

Planned use: Large-scale SfM; contaminated matches; advanced transfer.

Outside the core scope

Fall 2025 Quizzes 7 (ICP) and 8 (meshes) are recorded in the source register and excluded from Lectures 1–6.

GLOSSARY

Terms are added as their explanations are written. First-use definitions, glossary terms, and PDF index entries share stable references.

Homogeneous coordinates
: Coordinates representing a projective point by a nonzero vector up to nonzero scale. See *Notation and coordinate conventions* and (1) for the definition and comparison procedure.

Coordinate frame
: A reference origin and axes used to express coordinates. Transform directions are specified in *Notation and coordinate conventions*; detailed geometry is developed in *Reference frames and two worked quiz problems*.

Computer vision
: Inferring scene properties from images. See *Why reconstruct a world from photographs?* for the inverse-problem viewpoint.

Structure from motion
: Joint recovery of camera motion and scene structure from corresponding image measurements. See *Why reconstruct a world from photographs?*; full estimation follows in Lecture 6.

Depth map
: An image-indexed array of depth measurements or estimates. Its depth convention and camera model must be specified. See *Image formation: where does a 3D point appear?*.

Point cloud
: A collection of 3D point positions, often expressed in one common frame. See *Example 1.16*.

Projective plane
: Nonzero 3-vectors identified under nonzero scale, representing finite and ideal 2D points. See *Points, lines, and the meaning of infinity*.

Incidence
: A point lying on a line or plane, tested through a zero dot product of homogeneous coordinates and coefficients. See (1.1).

Ideal point
: A projective point with last homogeneous coordinate zero; also called a point at infinity. See (1.3).

Line at infinity
: The projective line containing all ideal points of a plane. See *Points, lines, and the meaning of infinity*.

Vanishing point
: An image location corresponding to a scene direction, often where projected parallel scene lines intersect. It need not be an ideal point of the image plane. See *Points, lines, and the meaning of infinity*.

Horizon
: The image of a scene plane's line at infinity. See *Points, lines, and the meaning of infinity*.

Degrees of freedom
: The number of independent continuous parameters. Matrix entries can be constrained or equivalent under scale. See (1.6) and *Rotations: several descriptions of the same motion*.

Nullspace
The vectors mapped to zero by a matrix. Homogeneous geometry seeks a nonzero solution, often unique up to scale. See (1.10).

Plücker coordinates
A homogeneous representation of a spatial line; for a finite line the direction/moment pair is $(d, p \times d)$ with $d^T m = 0$. See (1.12).

Reciprocal product
A scalar formed from two valid direction/moment line representations; it tests their coplanarity and supports a shortest-distance formula. See (1.13).

Surface normal
A direction perpendicular to local surface tangent directions. An oriented face normal requires a winding or outward convention. See *Example 1.12*.

Rotation matrix
A real orthogonal matrix with determinant $+1$ representing a proper rotation. See (1.15).

Axis–angle
A rotation representation consisting of a unit axis and an angle. Extraction needs special handling near zero and a half-turn. See (1.17).

Quaternion
A scalar–vector algebraic object; unit quaternions represent rotations under a declared multiplication and active/passive convention. See (1.19).

Intrinsic rotation
A rotation about an axis attached to the moving body. It is unrelated to camera intrinsic calibration despite the shared word. See (1.16).

Extrinsic rotation
A rotation about a fixed reference axis. See (1.16).

Gimbal lock
A singular three-angle description in which independent parameter effects coincide. The physical orientation still exists. See *Rotations: several descriptions of the same motion*.

Rigid transformation
A proper rotation and translation preserving distances and angles. See (1.20).

Circular points
Two complex ideal points shared by all Euclidean circles in a plane, encoding metric structure projectively. These are a vocabulary preview in *Move points without losing the geometry*.

Absolute conic
A complex conic on the plane at infinity encoding the 3D Euclidean metric. Its role in metric upgrades is deferred to Lecture 6; see *Reference frames and two worked quiz problems* for the hierarchy context.

Pinhole camera
An ideal central-projection model in which rays pass through one camera center. See (1.23).

Principal point
The pixel location where the optical axis intersects the ideal image plane. It is not automatically the pixel origin. See (1.24).

Intrinsic parameters
: Camera calibration parameters that map normalized camera coordinates to pixels, including focal entries, skew, and principal point. See (1.24).

Extrinsic parameters
: Rotation and translation linking world and camera frames, with a declared transform direction. See *Image formation: where does a 3D point appear?*.

Backprojection
: Mapping a pixel to a camera ray, then using specified depth to recover a 3D point. See (1.25).

Axial depth
: The point's coordinate along the camera's forward axis; it differs from Euclidean range along the ray. See (1.25).

Homography
: An invertible projective mapping between planes, represented by a nonsingular 3×3 matrix up to scale. See (2.4) and *Homographies: straighten a photographed plane*.

Cross-ratio
: An ordered ratio of four collinear positions preserved by projective mappings. See (2.6).

Invariant
: A property unchanged by a specified transformation family. See *Homographies: straighten a photographed plane*.

SLERP
: Spherical linear interpolation of unit quaternions, giving an orientation path with constant angular speed when parameterized uniformly. See (2.3).

Inverse warping
: Filling each output pixel by looking up a source coordinate through the inverse geometric mapping. See (2.7).

Bilinear interpolation
: Blending four neighboring grid samples using horizontal and vertical fractional-coordinate weights. See (2.8).

Direct linear transformation
: DLT: estimating a projective mapping from a homogeneous linear system. See (2.9); coordinate normalization is in (2.10).

Inlier
: A measurement consistent with a model within a specified residual threshold. An outlier fails that test. See *Estimate a reliable mapping*.

RANSAC
: Random sample consensus: fit small random subsets, score their support, and refit a selected consensus. See (2.11) for sampling assumptions.

Radial distortion
: Lens-induced coordinate displacement along directions from a distortion center. See (2.12), including mapping-direction conventions.

Epipolar line
: The image of an epipolar plane; a known point in one view restricts its counterpart to this line in the other.

See (2.13).

Epipole
The image of the other camera's center, shared by the epipolar lines of a view. See *Distortion and two-view geometry*.

Fundamental matrix
A rank-two 3×3 matrix mapping image points to epipolar lines in another view, defined up to scale. See (2.13).

Rectification
Warping a pair of images so corresponding epipolar lines lie on matching horizontal rows. See (2.16).

The planned text follows the six supplied CS 532 lecture PDFs and the quiz-to-lecture mapping in `quizByLec.md`. The lecture decks identify Enrique Dunn and credit additional contributors on individual sections. Preserve those specific credits when adapting their material; do not infer authorship or reuse permission from filenames alone.

Lecture 1 gives verified source ranges and identifies original versus adapted examples. The geometry section credits M. Pollefeys and D. Cappelleri (PDF page 62); image formation credits John Oliensis (PDF page 104). The notation and matrix-dimension examples in this foundation edition are original explanatory examples.

The pixel-axis diagram in the notation guide is an original project illustration with an editable TikZ source and paired SVG/PDF assets.

Lecture 1 includes three embedded image assets extracted without retouching from the supplied deck: the railway photograph (PDF page 68), fountain input (page 50), and fountain depth output (page 51). Their original individual creators are not identified on those slides; this source uncertainty is retained in the captions and figure register. The source deck is preserved. Twelve additional geometric/workflow diagrams are original project drawings, with paired SVG/PDF and editable TikZ sources. A repository's text license does not automatically establish the reuse permissions of every embedded source figure.

Reading list named in the lecture

The deck's PDF pages 9–10 name Richard Szeliski's *Computer Vision: Algorithms and Applications* (2010), David M. Mount's CMSC 754 computational-geometry lecture notes (2012), and Xiang Gao and Tao Zhang's *Introduction to Visual SLAM: From Theory to Practice*. These are source-listed further reading; the present chapter was developed from the supplied slides and quizzes rather than claiming those books were consulted in full.

Lecture 2

`cs532_f25_Week2.pdf` credits M. Pollefeys and D. Cappelleri for geometry (PDF page 2), John Oliensis for image formation (51), Steve Seitz for warping (72), and R. Hartley, A. Zisserman, and M. Pollefeys for estimation, distortion, and two-view geometry (82, 95, 105, 111).

The basketball-court photograph is extracted from PDF page 71, printed slide 72. Its individual photographer and reuse terms are not identified there. The annotated source/rectangular-warp comparison is computed locally from four disclosed approximate corner coordinates. The chosen output aspect ratio is illustrative. Four additional original vector diagrams explain quiz intersections, warp sampling, consensus, and rectified epipolar lines. Editable sources and paired print/web assets are retained in the figure register.

These source issues are flagged by the supplied study notes. Detailed corrections will be independently checked when the relevant worked examples are written.

Source	Issue	Planned treatment
Fall 2025 Quiz 2	A line-coefficient vector is described as a point	Verify arithmetic, matrix rank, and the scalar epipolar constraint in Lecture 4
Fall 2025 Quiz 5	A planar grid supports a full-scene metric-upgrade claim	State calibration/view assumptions and distinguish plane from full-scene recovery in Lecture 6
`cs532f15_quiz4.pdf`, `cs532f15_quiz9.pdf`	Filenames suggest 2015; printed headers identify Spring 2019	Retain both names in citations
`cs532s20_quiz3.pdf`	Printed header mixes years/quiz labels	Record the filename and printed wording

All source-solution computations remain subject to independent checking. Relevance scores describe slide relevance, not exam probabilities.

Lecture 1: verified qualifications and convention translations

- **Projective-plane shorthand (PDF page 71):** nonzero homogeneous vectors must be identified under common scale; they are not distinct projective points. See *Points, lines, and the meaning of infinity*.
- **Quaternion convention (PDF page 92):** the source sandwich is $q^{-1}Pq$. This book uses active Hamilton rotation qPq^{-1} and demonstrates the inverse relationship explicitly. Conjugation alone gives the inverse only for unit quaternions. See *Rotations: several descriptions of the same motion*.
- **Camera symbols (PDF pages 111–113):** camera-to-world pose and world-to-camera extrinsics use inverse transformations. The book translates the slide's notation and clarifies that projection maps 3D to 2D; its inverse without depth is a ray. See *Image formation: where does a 3D point appear?*.
- **Plane uniqueness (Fall 2025 Quiz 1 Q4):** camera centers and the chosen object-frame origin determine a unique plane only when noncollinear. See *Example 1.11*.
- **Nullspace wording (PDF pages 83–85):** solve the right nullspace of the stacked incidence matrix, not "the nullspace of" the unknown point/plane. See *Points, planes, and lines in 3D*.

Numerical companions are checked by `scripts/check_lecture01_math.py`. These qualifications do not imply that every flagged issue in later lectures has already been checked.

Lecture 2: verified corrections and assumptions

- **Matrix-to-quaternion formula, PDF page 49:** the scalar component needs $w = \frac{1}{2}\sqrt{1 + \operatorname{tr} R}$. The slide omits $+1$; identity, quarter-turn, and cyclic-axis checks expose the error. Near a half-turn, use a component branch instead of dividing by w. See *Quaternions: convert without changing the rotation*.
- **Quaternion convention, PDF pages 43, 48–49:** the sandwich and quaternion-to-matrix slide use the inverse/passive convention relative to the book, while the displayed skew-entry extraction uses active signs. The book uses active Hamilton formulas consistently and verifies matrix round trips.
- **Geometry shorthand, PDF pages 11, 23–25:** identify homogeneous vectors under nonzero scale; compute the right nullspace of the incidence coefficient matrix, not of the unknown vector. These repeat Lecture 1's qualifications.
- **Camera direction, PDF page 60:** the displayed projection goes from 3D to 2D despite the prose saying 2D to 3D. Without depth, inverse projection gives a ray. See *Foundation quiz practice*.
- **Radius notation, PDF page 109:** the displayed polynomial and radius definition mix squared-radius conventions. The book defines r^2 explicitly and states the correction direction and coefficient units. See (2.12).
- **Fundamental-matrix uniqueness, PDF page 124:** uniqueness is up to scale for a fixed nondegenerate camera pair; a finite match set can be ambiguous. Coincident camera centers do not yield a nonzero fundamental matrix. See *Distortion and two-view geometry*.
- **Quiz degeneracy, Spring 2020 Quiz 1 P1:** repeated points make the cross-product line undefined, although the determinant still correctly indicates linear dependence and existence of a containing line. P3 requires $d > 0$ for distinct vertical lines. See *Foundation quiz practice*.

Lecture 2's numerical examples are checked by `scripts/check_lecture02_math.py`.

INDEX

A

Absolute conic, **83**
Axial depth, **84**
axial depth, 44
Axis–angle, **83**
axis–angle, 33

B

Backprojection, **84**
backprojection, 44
Bilinear interpolation, **84**
bilinear interpolation, 60

C

Circular points, **83**
collinearity, 47
Computer vision, **82**
computer vision
    inverse problem, 11
Coordinate frame, **82**
Cross–ratio, **84**
cross–ratio, 58

D

Degrees of freedom, **82**
Depth map, **82**
Direct linear transformation, **84**
direct linear transformation, 65

E

Epipolar line, **84**
epipolar line, 69
Epipole, **85**
epipole, 69
Extrinsic parameters, **84**
extrinsic parameters, 44
Extrinsic rotation, **83**
extrinsic rotation, 33

F

Fundamental matrix, **85**
fundamental matrix, 69

G

geometric reconstruction, 11
Gimbal lock, **83**
gimbal lock, 33

H

Homogeneous coordinates, **82**
homogeneous coordinates
    scale equivalence, 4
Homography, **84**
homography, 58
Horizon, **82**

I

Ideal point, **82**
Incidence, **82**
incidence, 15
Inlier, **84**
inlier, 65
Intrinsic parameters, **84**
intrinsic parameters, 44
Intrinsic rotation, **83**
intrinsic rotation, 33
Invariant, **84**
invariant, 58
Inverse warping, **84**
inverse warping, 60

L

Line at infinity, **82**
line at infinity, 15

N

normal
    outward orientation, 38

normalized DLT, 65
Nullspace, **83**
nullspace, 22

P

perspective projection, 44
Pinhole camera, **83**
pinhole camera, 44
plane
   normal, 22
Plücker coordinates, 22, **83**
point at infinity, 15
Point cloud, **82**
Principal point, **83**
principal point, 44
Projective plane, **82**
projective plane, 15

Q

Quaternion, **83**
quaternion, 33

R

Radial distortion, **84**
radial distortion, 69
RANSAC, 65, **84**
Reciprocal product, **83**
Rectification, **85**
rectification, 69
reference frame, 38
Rigid transformation, **83**
rigid transformation, 38
Rotation matrix, **83**
rotation matrix, 33

S

semantic information, 11
SLERP, 56, **84**
Structure from motion, **82**
Surface normal, **83**
surface normal, 22

T

transform composition, 38

V

Vanishing point, **82**